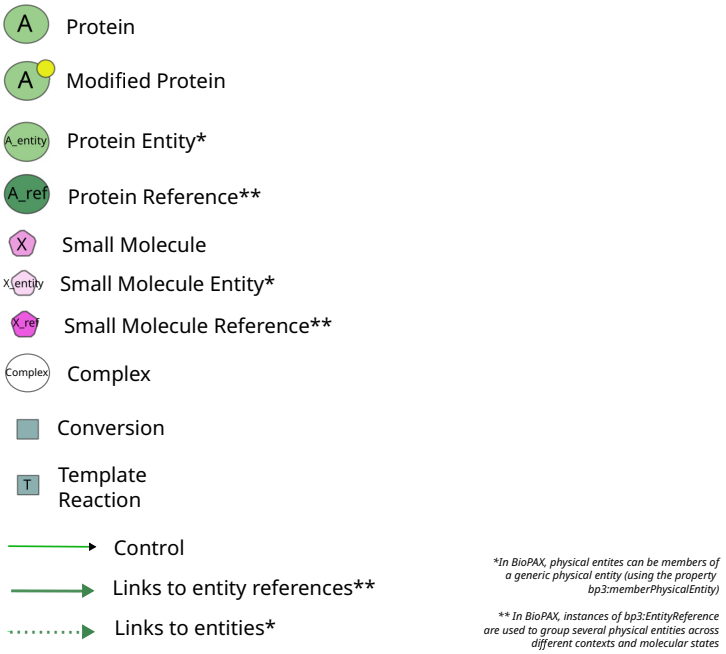
Clarifying BioPAX to SIF abstraction rules

Supplementary Material S-1: BioPAX to SIF SPARQL queries for the 3 meanings of description of SIF rules

Caption of diagrams and detail of meanings constraints

For the three meanings of description of SIF rules, we adopted the style of the BioPAX to SIF abstraction rules diagrams presented on the PathwayCommons website [1].

- Meaning 1 diagrams correspond to the same diagrams and textual descriptions presented on the PathwayCommons website.
- Meaning 2 diagrams recapitulate the constraints of the Paxtools patterns functions descriptions from the Paxtools code for each SIF rule. In these descriptions, controlling entities are reached by their entity reference. **For the sake of consistency, we assumed that controlled entities would also be reached through their reference.**
- Meaning 3 diagrams recapitulate the constraints of the Paxtools patterns code for each SIF rule. The main added constraints in the code are that controlling entities can be part of complexes, that general entities can group other entities and that ubiquitous small molecules are excluded from the results (use of a blacklist).



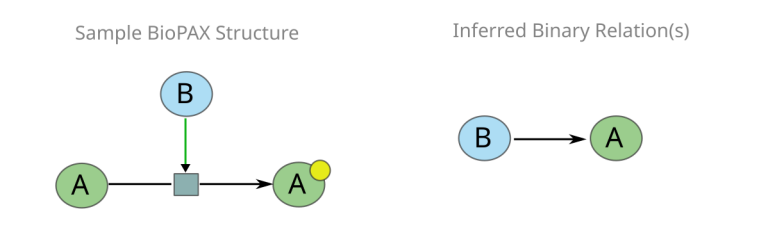
[1] https://www.pathwaycommons.org/pc2/formats

1 Meaning 1 of SIF rules description: SIF diagrams and associated textual descriptions from PathwayCommons

1.1 controls-state-change-of

SIF diagram and textual description from PathwayCommons

First protein controls a reaction that changes the state of the second protein.



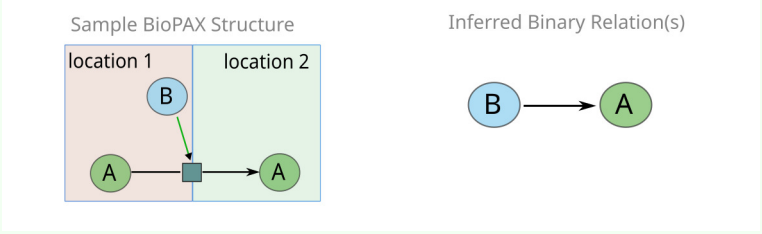
Meaning 1 SPARQL query

```
CONSTRUCT {
  ?enzyme abstraction:ControlsStateChangeOf ?prot
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?catalysis bp3:controlled ?reaction .
  ?catalysis (rdf:type/rdfs:subClassOf*) bp3:Control .
  ?catalysis bp3:controller ?enzyme .
  ?enzyme rdf:type bp3:Protein .
  ?reaction bp3:left ?protLeft .
  ?protLeft rdf:type bp3:Protein .
  ?reaction bp3:right ?protRight .
  ?protRight rdf:type bp3:Protein .
  ?protRight bp3:feature ?feature .
  ?feature rdf:type/(rdfs:subClassOf*) bp3:EntityFeature .
}
```

1.2 controls-transport-of

SIF diagram and textual description from PathwayCommons

First protein controls a reaction that changes the cellular location of the second protein.



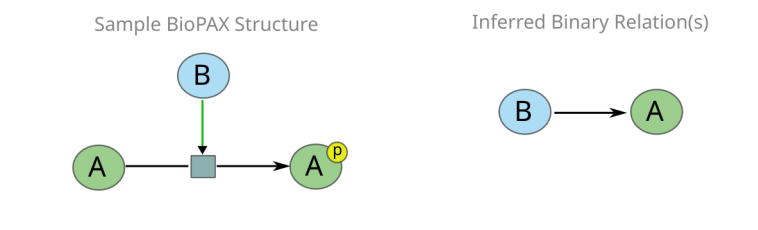
Meaning 1 SPARQL query

```
CONSTRUCT {
  ?enzyme abstraction:ControlsTransportOf ?protLeft
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction bp3:left ?protLeft .
  ?protLeft rdf:type bp3:Protein .
  ?protLeft bp3:entityReference ?protRef .
  ?reaction bp3:right ?protRight .
  ?protRight rdf:type bp3:Protein .
  ?protRight bp3:entityReference ?protRef .

  # Define cellular locations for Protein 1 and Protein 2
  ?protLeft bp3:cellularLocation ?cellularLocVocab1 .
  ?protRight bp3:cellularLocation ?cellularLocVocab2 .

  ?catalysis bp3:controlled ?reaction .
  ?catalysis (rdf:type/rdfs:subClassOf*) bp3:Control .
  ?catalysis bp3:controller ?enzyme .
  ?enzyme rdf:type bp3:Protein .

  # Ensure cellular locations of protein 1 and protein 2 are different
  FILTER (?cellularLocVocab1 != ?cellularLocVocab2)
}
```

1.3 controls-phosphorylation-of

SIF diagram and textual description from PathwayCommons

First protein controls a reaction that changes the phosphorylation status of the second protein.



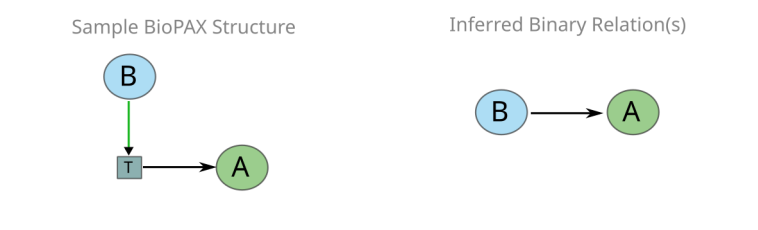
Meaning 1 SPARQL query

```
CONSTRUCT {
?enzyme abstraction:ControlsPhosphorylationOf ?protLeft
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?catalysis bp3:controlled ?reaction .
  ?catalysis rdf:type/(rdfs:subClassOf*) bp3:Control .
  ?catalysis bp3:controller ?enzyme .
  ?enzyme rdf:type bp3:Protein .

  ?reaction bp3:left ?protLeft .
  ?protLeft rdf:type bp3:Protein .
  ?protLeft bp3:entityReference ?protRef .
  ?reaction bp3:right ?protRight .
  ?protRight rdf:type bp3:Protein .
  ?protRight bp3:entityReference ?protRef .

  # Specify that ?protRight is phosphorylated
  ?protRight bp3:feature ?feature .
  ?feature rdf:type bp3:ModificationFeature .
  ?feature bp3:modificationType ?modificationType .
  ?modificationType rdf:type bp3:SequenceModificationVocabulary .
  ?modificationType bp3:term ?modificationTerm .
  FILTER(CONTAINS(LCASE(?modificationTerm), "phospho"))
}
```

1.4 controls-expression-of

SIF diagram and textual description from PathwayCommons

First protein controls a conversion or a template reaction that changes expression of the second protein.



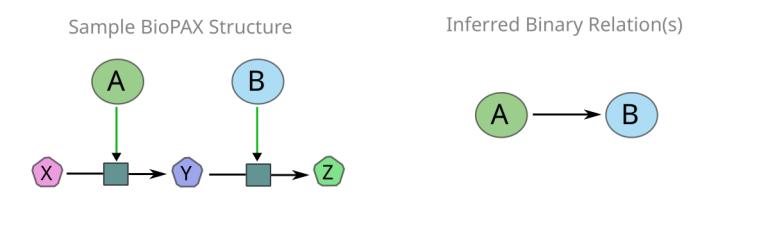
Meaning 1 SPARQL query

```
CONSTRUCT {
  ?enzyme abstraction:ControlsExpressionOf ?product
}
WHERE {
  # Case where the reaction is a Conversion
  {
    VALUES ?NucleicAcid { bp3:Dna bp3:Rna }
    ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
    ?reaction bp3:left ?left .
    ?left rdf:type ?NucleicAcid .
    ?reaction bp3:right ?product .
    ?product rdf:type bp3:Protein .
    ?catalysis bp3:controlled ?reaction .
    ?catalysis rdf:type/(rdfs:subClassOf*) bp3:Control .
    ?catalysis bp3:controller ?enzyme .
    ?enzyme rdf:type bp3:Protein .
  }
  UNION
  # Case where the reaction is a TemplateReaction
  {
    ?reaction rdf:type/(rdfs:subClassOf*) bp3:TemplateReaction .
    ?reaction bp3:product ?product .
    ?product rdf:type bp3:Protein .
    ?catalysis bp3:controlled ?reaction .
    ?catalysis rdf:type bp3:TemplateReactionRegulation .
    ?catalysis bp3:controller ?enzyme .
    ?enzyme rdf:type bp3:Protein .
  }
}
```

1.5 catalysis-precedes

SIF diagram and textual description from PathwayCommons

First protein controls a reaction whose output molecule is input to another reaction controled by the second protein.



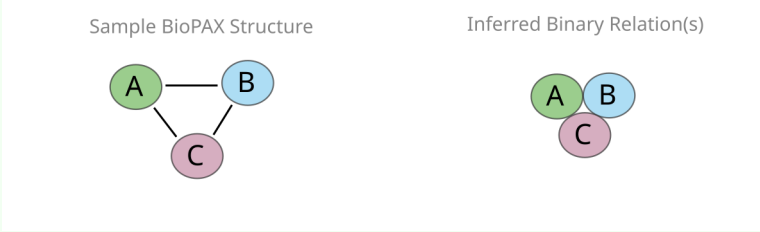
Meaning 1 SPARQL query

```
CONSTRUCT {
   ?enzymeA abstraction:CatalysisPrecedes ?enzymeB
}
WHERE {
  ?reaction1 rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?catalysis1 bp3:controlled ?reaction1 .
  ?reaction1 bp3:left ?leftMolecule1 .
  ?leftMolecule1 rdf:type bp3:SmallMolecule .
  ?reaction1 bp3:right ?connectingMolecule .
  ?connectingMolecule rdf:type bp3:SmallMolecule .
  ?catalysis1 (rdf:type/rdfs:subClassOf*) bp3:Control .
  ?catalysis1 bp3:controller ?enzymeA .
  ?enzymeA rdf:type bp3:Protein .

  ?reaction2 rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction2 bp3:left ?connectingMolecule .
  ?reaction2 bp3:right ?rightMolecule2 .
  ?rightMolecule2 rdf:type bp3:SmallMolecule .
  ?catalysis2 bp3:controlled ?reaction2 .
  ?catalysis2 (rdf:type/rdfs:subClassOf*) bp3:Control .
  ?catalysis2 bp3:controller ?enzymeB .
  ?enzymeB rdf:type bp3:Protein .
}
```

1.6 in-complex-with

SIF diagram and textual description from PathwayCommons

Proteins are members of the same complex.



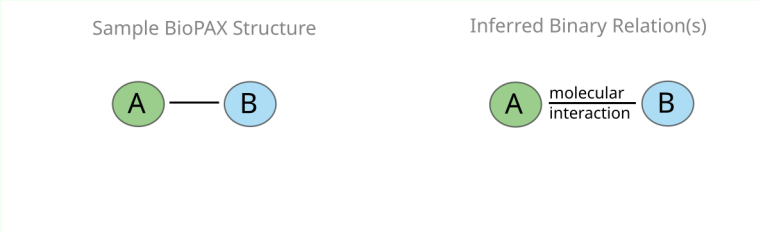
Meaning 1 SPARQL query

```
CONSTRUCT {
   ?prot1 abstraction:InComplexWith ?prot2
}
WHERE {
  # Case 1: Complex with several different proteins
  {
?complex rdf:type/(rdfs:subClassOf*) bp3:Complex .
?complex bp3:component ?prot1 .
?complex bp3:component ?prot2 .
?prot1 rdf:type bp3:Protein .
?prot2 rdf:type bp3:Protein .
# Avoid self-pairing and duplicate pairs
FILTER (STR(?prot1) < STR(?prot2))
  }
  UNION
  # Case 2: Same protein with stoichiometry > 1
  {
?complex rdf:type/(rdfs:subClassOf*) bp3:Complex .
# Check for stoichiometry > 1
?complex bp3:componentStoichiometry ?stoich .
?stoich bp3:physicalEntity ?entity1 .
?stoich bp3:stoichiometricCoefficient ?coeff .
FILTER (?coeff > 1)
    ?complex bp3:component ?prot1 .
?prot1 rdf:type bp3:Protein .
# For self-pairing, use the same entity
BIND (?prot1 AS ?prot2)
  }
}
```

1.7 interacts-with

SIF diagram and textual description from PathwayCommons

Proteins are participants of the same MolecularInteraction.



Meaning 1 SPARQL query

```
CONSTRUCT {
  ?participant1 abstraction:InteractsWith ?participant2
}
WHERE {
  VALUES ?participantType { bp3:Protein bp3:SmallMolecule }
  ?MolecularInteraction rdf:type/(rdfs:subClassOf*) bp3:MolecularInteraction .
  ?MolecularInteraction bp3:participant ?participant1 .
  ?participant1 rdf:type ?participantType .
  ?MolecularInteraction bp3:participant ?participant2 .
  ?participant2 rdf:type ?participantType .
  # Avoid self-pairing and duplicate pairs
  FILTER (STR(?participant1) < STR(?participant2))
}
```

1.8 neighbor-of

SIF diagram and textual description from PathwayCommons

Proteins are participants or controlers of the same interaction.

Meaning 1 SPARQL query

```
CONSTRUCT {
   ?prot1 abstraction:NeighborOf ?prot2
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Interaction .
  ?reaction ((^bp3:controlled/bp3:controller)|bp3:left|bp3:right) ?prot1 .
  ?prot1 rdf:type bp3:Protein .
  ?reaction ((^bp3:controlled/bp3:controller)|bp3:left|bp3:right) ?prot2 .
  ?prot2 rdf:type bp3:Protein .
  # Avoid self-pairing and duplicate pairs
  FILTER (STR(?prot1) < STR(?prot2))
}
```

1.9 consumption-controlled-by

SIF diagram and textual description from PathwayCommons

The small molecule is consumed by a reaction that is controlled by a protein

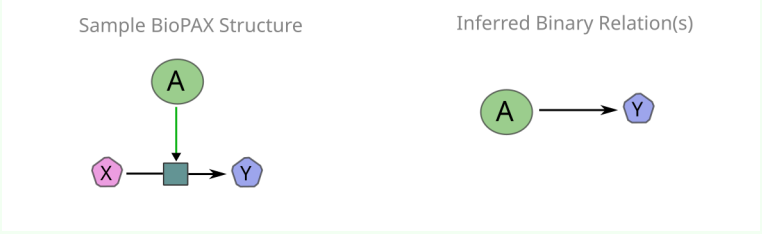
Meaning 1 SPARQL query

```
CONSTRUCT {
  ?reactant abstraction:ConsumptionControlledBy ?enzyme
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction bp3:left ?reactant .
  ?reactant rdf:type bp3:SmallMolecule .
  ?reaction bp3:right ?product .
  ?product rdf:type bp3:SmallMolecule .
  ?catalysis bp3:controlled ?reaction .
  ?catalysis bp3:controller ?enzyme .
  ?enzyme rdf:type bp3:Protein .
}
```

1.10 controls-production-of

SIF diagram and textual description from PathwayCommons

The protein controls a reaction of which the small molecule is an output.



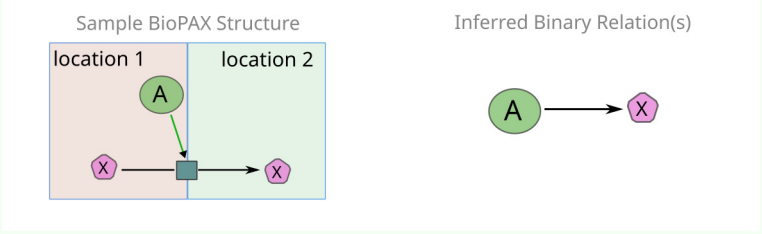
Meaning 1 SPARQL query

```
CONSTRUCT {
  ?enzyme abstraction:ControlsProductionOf ?product
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction bp3:left ?reactant .
  ?reactant rdf:type bp3:SmallMolecule .
  ?reaction bp3:right ?product .
  ?product rdf:type bp3:SmallMolecule .
  ?catalysis bp3:controlled ?reaction .
  ?catalysis bp3:controller ?enzyme .
  ?enzyme rdf:type bp3:Protein .
}
```

1.11 controls-transport-of-chemical

SIF diagram and textual description from PathwayCommons

The protein controls a reaction that changes cellular location of the small molecule.



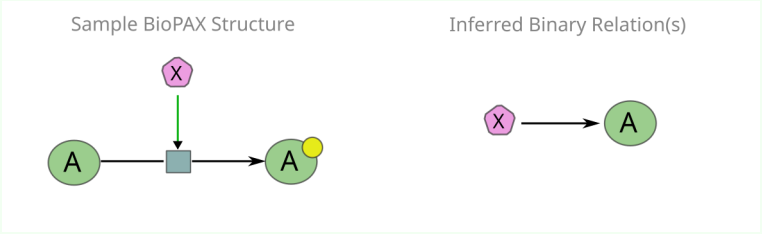
Meaning 1 SPARQL query

```
CONSTRUCT {
  ?enzyme abstraction:ControlsTransportOfChemical ?SmallMolecule1
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction bp3:left ?SmallMolecule1 .
  ?SmallMolecule1 rdf:type bp3:SmallMolecule .
  ?SmallMolecule1 bp3:cellularLocation ?cellularLocVocab1 .
  ?SmallMolecule1 bp3:entityReference ?SmallMoleculeRef .

  ?reaction bp3:right ?SmallMolecule2 .
  ?SmallMolecule2 rdf:type bp3:SmallMolecule .
  ?SmallMolecule2  bp3:cellularLocation ?cellularLocVocab2 .
  ?SmallMolecule2 bp3:entityReference ?SmallMoleculeRef .

  ?catalysis bp3:controlled ?reaction .
  ?catalysis rdf:type/(rdfs:subClassOf*) bp3:Control .
  ?catalysis bp3:controller ?enzyme .
  ?enzyme rdf:type bp3:Protein .

  FILTER(?cellularLocVocab1 != ?cellularLocVocab2)
}
```

1.12 chemical-affects

SIF diagram and textual description from PathwayCommons

A small molecule has an effect on the protein state.



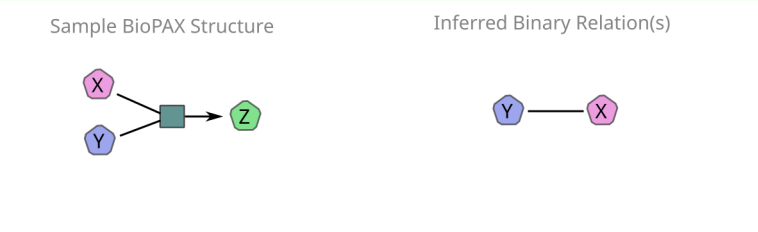
Meaning 1 SPARQL query

```
CONSTRUCT {
  ?chemicalCatalyzer abstraction:ChemicalAffects ?protein .
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction bp3:left ?protLeft .
  ?protLeft rdf:type bp3:Protein .
  ?protLeft bp3:entityReference ?protRef .
  ?reaction bp3:right ?protRight .
  ?protRight rdf:type bp3:Protein .
  # modification of the protein state
  ?protRight bp3:feature ?feature .
  ?feature rdf:type/(rdfs:subClassOf*) bp3:EntityFeature .
  ?protRight bp3:entityReference ?protRef .
  ?catalysis bp3:controlled ?reaction .
  ?catalysis rdf:type/(rdfs:subClassOf*) bp3:Control .
  ?catalysis bp3:controller ?chemicalCatalyzer .
  ?chemicalCatalyzer rdf:type bp3:SmallMolecule .
}
```

1.13 reacts-with

SIF diagram and textual description from PathwayCommons

Small molecules are input to a biochemical reaction.



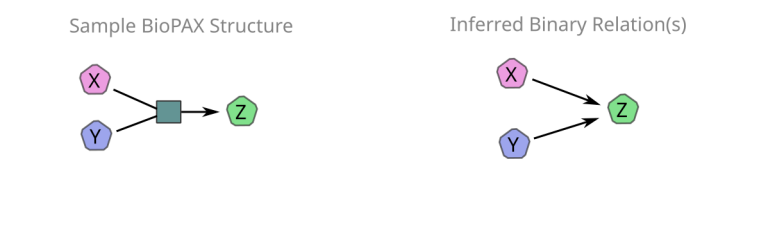
Meaning 1 SPARQL query

```
CONSTRUCT {
  ?smallMolecule1 abstraction:ReactsWith ?smallMolecule2 .
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:BiochemicalReaction .
  ?reaction bp3:left ?smallMolecule1 .
  ?smallMolecule1 rdf:type bp3:SmallMolecule .
  ?reaction bp3:left ?smallMolecule2 .
  ?smallMolecule2 rdf:type bp3:SmallMolecule .
  FILTER (?smallMolecule1 != ?smallMolecule2)
}
```

1.14 used-to-produce

SIF diagram and textual description from PathwayCommons

A reaction consumes a small molecule to produce another small molecule.



Meaning 1 SPARQL query

```
CONSTRUCT {
  ?smallMolecule1 abstraction:UsedToProduce ?smallMolecule2 .
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction bp3:left ?smallMolecule1 .
  ?reaction bp3:right ?smallMolecule2 .
  ?smallMolecule1 rdf:type bp3:SmallMolecule .
  ?smallMolecule2 rdf:type bp3:SmallMolecule .
  FILTER (?smallMolecule1 != ?smallMolecule2)
}
```

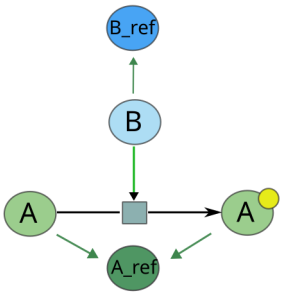
2 Meaning 2 of SIF rules description: Paxtools patterns functions descriptions

2.1 controls-state-change-of

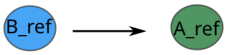
Paxtools pattern `controlsStateChange()` description

Pattern for a EntityReference has a member PhysicalEntity that is controlling a state change reaction of another EntityReference.

Sample BioPAX structure



Inferred binary relation(s)



Meaning 2 SPARQL query

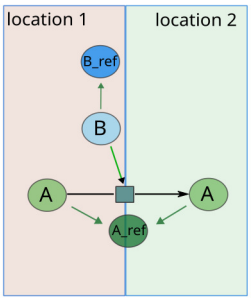
```
CONSTRUCT {
  ?enzymeRef abstraction:ControlsStateChangeOf ?protRef
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?catalysis bp3:controlled ?reaction .
  ?catalysis (rdf:type/rdfs:subClassOf*) bp3:Control .
  ?catalysis bp3:controller ?enzyme .
  ?enzyme rdf:type bp3:Protein .
  ?enzyme bp3:entityReference ?enzymeRef .
  ?enzymeRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .

  ?reaction bp3:left ?protLeft .
  ?protLeft rdf:type bp3:Protein .
  ?protLeft bp3:entityReference ?protRef .
  ?reaction bp3:right ?protRight .
  ?protRight rdf:type bp3:Protein .
  ?protRight bp3:entityReference ?protRef .
  ?protRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .

  # modification of the right protein
  ?protRight bp3:feature ?feature .
  ?feature rdf:type/(rdfs:subClassOf*) bp3:EntityFeature .
  FILTER (?enzymeRef != ?protRef)
}
```

2.2 controls-transport-of

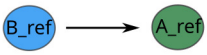
Paxtools pattern `controlsTransport()` description

Pattern for a ProteinReference has a member PhysicalEntity that is controlling a transportation of another ProteinReference.

Sample BioPAX structure



Inferred binary relation(s)



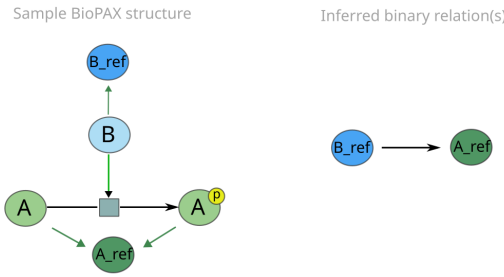
Meaning 2 SPARQL query

```
CONSTRUCT {
  ?enzymeRef abstraction:ControlsTransportOf ?protRef
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction bp3:left ?protLeft .
  ?protLeft rdf:type bp3:Protein .
  ?protLeft bp3:entityReference ?protRef .
  ?reaction bp3:right ?protRight .
  ?protRight rdf:type bp3:Protein .
  ?protRight bp3:entityReference ?protRef .
  ?protRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .

  # Define cellular locations for Protein 1 and Protein 2
  ?protLeft bp3:cellularLocation ?cellularLocVocab1 .
  ?protRight bp3:cellularLocation ?cellularLocVocab2 .

  ?catalysis bp3:controlled ?reaction .
  ?catalysis (rdf:type/rdfs:subClassOf*) bp3:Control .
  ?catalysis bp3:controller ?enzyme .
  ?enzyme rdf:type bp3:Protein .
  ?enzyme bp3:entityReference ?enzymeRef .
  ?enzymeRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .

  # Ensure cellular locations of protein 1 and protein 2 are different
  FILTER (?cellularLocVocab1 != ?cellularLocVocab2)
  # Ensure the enzyme is not the transported protein
  FILTER (?enzymeRef != ?protRef)
}
```

2.3 controls-phosphorylation-of

Paxtools pattern `controlsPhosphorylation()` description

No function description in Paxtools



Meaning 2 SPARQL query

```
CONSTRUCT {
?enzymeRef abstraction:ControlsPhosphorylationOf ?protRef
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?catalysis bp3:controlled ?reaction .
  ?catalysis rdf:type/(rdfs:subClassOf*) bp3:Control .
  ?catalysis bp3:controller ?enzyme .
  ?enzyme rdf:type bp3:Protein .
  ?enzyme bp3:entityReference ?enzymeRef .
  ?enzymeRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .

  ?reaction bp3:left ?protLeft .
  ?protLeft rdf:type bp3:Protein .
  ?protLeft bp3:entityReference ?protRef .
  ?reaction bp3:right ?protRight .
  ?protRight rdf:type bp3:Protein .
  ?protRight bp3:entityReference ?protRef .
  ?protRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .

  # Specify that ?protRight is phosphorylated
  ?protRight bp3:feature ?feature .
  ?feature rdf:type bp3:ModificationFeature .
  ?feature bp3:modificationType ?modificationType .
  ?modificationType rdf:type bp3:SequenceModificationVocabulary .
  ?modificationType bp3:term ?modificationTerm .
  FILTER(CONTAINS(LCASE(?modificationTerm), "phospho"))
  FILTER(?enzymeRef != ?protRef)
}
```

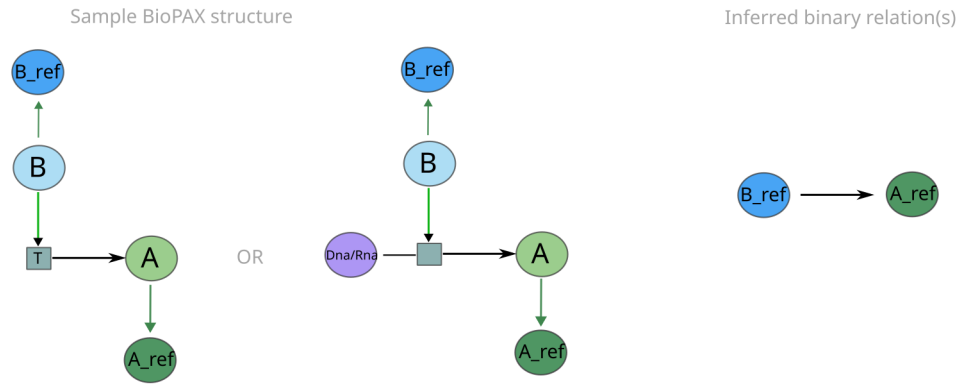
2.4 controls-expression-of

Paxtools pattern `controlsExpressionWithTemplateReac()` description

Finds transcription factors that trans-activate or trans-inhibit an entity.

Paxtools pattern `controlsExpressionWithConversion()` description

Finds the cases where transcription relation is shown using a Conversion instead of a TemplateReaction.



Meaning 2 SPARQL query

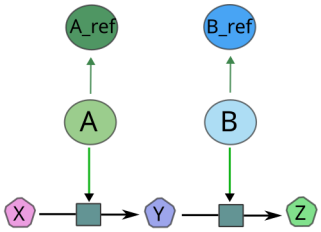
```
CONSTRUCT {
  ?enzymeRef abstraction:ControlsExpressionOf ?productRef
}
WHERE {
# Case where the reaction is a Conversion
  {
    VALUES ?NucleicAcid { bp3:Dna bp3:Rna }
    ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
    ?reaction bp3:left ?left .
    ?left rdf:type ?NucleicAcid .
    ?reaction bp3:right ?product .
    ?product rdf:type bp3:Protein .
    ?product bp3:entityReference ?productRef .
    ?productRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
    ?catalysis bp3:controlled ?reaction .
    ?catalysis rdf:type/(rdfs:subClassOf*) bp3:Control .
    ?catalysis bp3:controller ?enzyme .
    ?enzyme rdf:type bp3:Protein .
    ?enzyme bp3:entityReference ?enzymeRef .
    ?enzymeRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  }
  UNION
  # Case where the reaction is a TemplateReaction
  {
    ?reaction rdf:type/(rdfs:subClassOf*) bp3:TemplateReaction .
    ?reaction bp3:product ?product .
    ?product rdf:type bp3:Protein .
    ?product bp3:entityReference ?productRef .
    ?productRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
    ?catalysis bp3:controlled ?reaction .
    ?catalysis rdf:type bp3:TemplateReactionRegulation .
    ?catalysis bp3:controller ?enzyme .
    ?enzyme rdf:type bp3:Protein .
    ?enzyme bp3:entityReference ?enzymeRef .
    ?enzymeRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  }
  FILTER(?enzymeRef != ?productRef)
}
```

2.5 catalysis-precedes

Paxtools pattern `catalysisPrecedes()` description

Pattern for detecting two EntityReferences are controlling consecutive reactions, where output of one reaction is input to the other.

Sample BioPAX structure



Inferred binary relation(s)

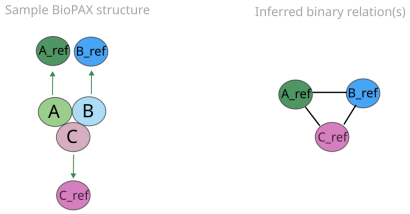
Meaning 2 SPARQL query

```
CONSTRUCT {
   ?enzymeRefA abstraction:CatalysisPrecedes ?enzymeRefB
}
WHERE {
  ?reaction1 rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction1 bp3:left ?leftMolecule1 .
  ?leftMolecule1 rdf:type bp3:SmallMolecule .
  ?reaction1 bp3:right ?connectingMolecule .
  ?connectingMolecule rdf:type bp3:SmallMolecule .
  ?catalysis1 bp3:controlled ?reaction1 .
  ?catalysis1 (rdf:type/rdfs:subClassOf*) bp3:Control .
  ?catalysis1 bp3:controller ?enzymeA .
  ?enzymeA rdf:type bp3:Protein .
  ?enzymeA bp3:entityReference ?enzymeRefA .
  ?enzymeRefA rdf:type/(rdfs:subClassOf*) bp3:EntityReference .

  ?reaction2 rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction2 bp3:left ?connectingMolecule .
  ?reaction2 bp3:right ?rightMolecule2 .
  ?rightMolecule2 rdf:type bp3:SmallMolecule .
  ?catalysis2 bp3:controlled ?reaction2 .
  ?catalysis2 (rdf:type/rdfs:subClassOf*) bp3:Control .
  ?catalysis2 bp3:controller ?enzymeB .
  ?enzymeB rdf:type bp3:Protein .
  ?enzymeB bp3:entityReference ?enzymeRefB .
  ?enzymeRefB rdf:type/(rdfs:subClassOf*) bp3:EntityReference .

  FILTER (?enzymeRefA != ?enzymeRefB)
}
```

2.6 in-complex-with

Paxtools pattern `inComplexWith()` description

Two proteins have states that are members of the same complex. Handles nested complexes and homologies. Also guarantees that relationship to the complex is through different direct complex members.



Meaning 2 SPARQL query

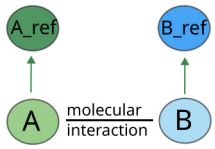
```
CONSTRUCT {
  ?prot1Ref abstraction:InComplexWith ?prot2Ref
}
WHERE {
  # Case 1: Complex with several different proteins
  {
?complex rdf:type/(rdfs:subClassOf*) bp3:Complex .
?complex (bp3:component|bp3:memberPhysicalEntity)*  ?prot1 .
?complex (bp3:component|bp3:memberPhysicalEntity)*  ?prot2 .
?prot1 rdf:type bp3:Protein .
?prot2 rdf:type bp3:Protein .
# Avoid self-pairing and duplicate pairs
FILTER (STR(?prot1) < STR(?prot2))
  }
  UNION
  # Case 2: Same protein with stoichiometry > 1
  {
?complex rdf:type/(rdfs:subClassOf*) bp3:Complex .
# Check for stoichiometry > 1
?complex bp3:componentStoichiometry ?stoich .
?stoich bp3:physicalEntity ?entity1 .
?stoich bp3:stoichiometricCoefficient ?coeff .
FILTER (?coeff > 1)
    ?complex bp3:component ?prot1 .
?prot1 rdf:type bp3:Protein .
# For self-pairing, use the same entity
BIND (?prot1 AS ?prot2)
  }
  ?prot1 bp3:entityReference ?prot1Ref .
  ?prot2 bp3:entityReference ?prot2Ref .
  ?prot1Ref rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  ?prot2Ref rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
}
```

2.7 interacts-with

Paxtools pattern `molecularInteraction()` description

Constructs a pattern where first and last molecules are participants of a MolecularInteraction.

Sample BioPAX structure



Inferred binary relation(s)

Meaning 2 SPARQL query

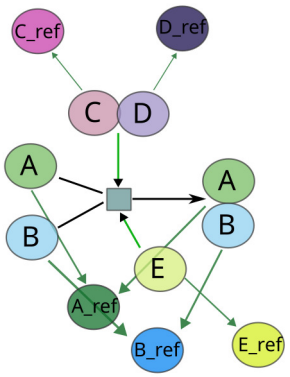
```
CONSTRUCT {
  ?participant1Ref abstraction:InteractsWith ?participant2Ref
}
WHERE {
  VALUES ?participantType { bp3:Protein bp3:SmallMolecule }
  ?MolecularInteraction rdf:type/(rdfs:subClassOf*) bp3:MolecularInteraction .
  ?MolecularInteraction bp3:participant ?participant1 .
  ?participant1 rdf:type ?participantType .
  ?MolecularInteraction bp3:participant ?participant2 .
  ?participant2 rdf:type ?participantType .

  ?participant1 bp3:entityReference ?participant1Ref .
  ?participant1Ref rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  ?participant2 bp3:entityReference ?participant2Ref .
  ?participant2Ref rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  FILTER (STR(?participant1Ref) < STR(?participant2Ref))
}
```

2.8 neighbor-of

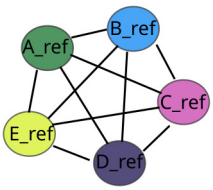
Paxtools pattern `neighborOf()` description

Constructs a pattern where first and last proteins are related through an interaction. They can be participants or controllers. No limitation.

Sample BioPAX structure



Inferred binary relation(s)



Meaning 2 SPARQL query

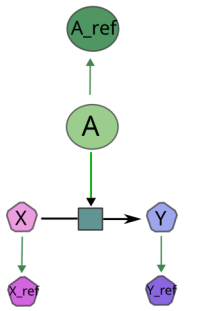
```
CONSTRUCT {
   ?prot1Ref abstraction:NeighborOf ?prot2Ref
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Interaction .
  ?reaction ((^bp3:controlled/bp3:controller)|bp3:left|bp3:right|bp3:participant) ?prot1 .
  ?prot1 rdf:type bp3:Protein .
  ?prot1 bp3:entityReference ?prot1Ref .
  ?prot1Ref rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  ?reaction ((^bp3:controlled/bp3:controller)|bp3:left|bp3:right|bp3:participant) ?prot2 .
  ?prot2 rdf:type bp3:Protein .
  ?prot2 bp3:entityReference ?prot2Ref .
  ?prot2Ref rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  FILTER (STR(?prot1Ref) < STR(?prot2Ref))
}
```

2.9 consumption-controlled-by

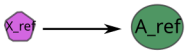
Paxtools pattern `controlsMetabolicCatalysis()` description

Pattern for a Protein controlling a reaction whose participant is a small molecule

Sample BioPAX structure



Inferred binary relation(s)



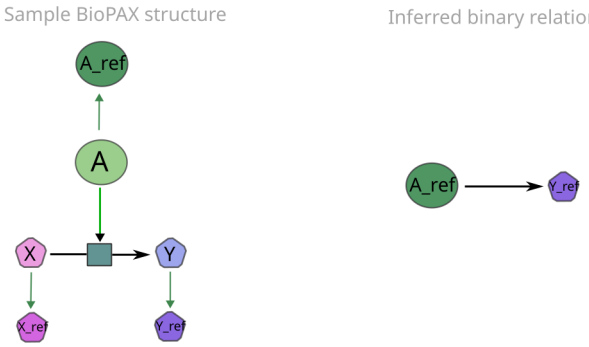
Meaning 2 SPARQL query

```
CONSTRUCT {
  ?reactantRef abstraction:ConsumptionControlledBy ?enzymeRef
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction bp3:left ?reactant .
  ?reactant rdf:type bp3:SmallMolecule .
  ?reactant bp3:entityReference ?reactantRef .
  ?reactantRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  ?reaction bp3:right ?product .
  ?product rdf:type bp3:SmallMolecule .
  ?catalysis bp3:controlled ?reaction .
  ?catalysis bp3:controller ?enzyme .
  ?enzyme rdf:type bp3:Protein .
  ?enzyme bp3:entityReference ?enzymeRef .
  ?enzymeRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
}
```

2.10 controls-production-of

Paxtools pattern description

There is no Paxtools Pattern for the SIF rule `controls-production-of`. Since this rule is based on the same pattern than the SIF rule `consumption-controlled-by`, we used the same logic for the meaning 2.



Meaning 2 SPARQL query

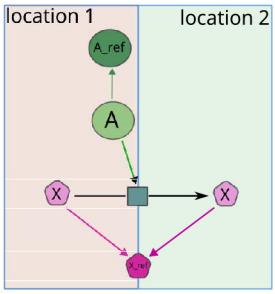
```
CONSTRUCT {
  ?enzymeRef abstraction:ControlsProductionOf ?productRef
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction bp3:left ?reactant .
  ?reactant rdf:type bp3:SmallMolecule .
  ?reaction bp3:right ?product .
  ?product rdf:type bp3:SmallMolecule .
  ?product bp3:entityReference ?productRef .
  ?productRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  ?catalysis bp3:controlled ?reaction .
  ?catalysis bp3:controller ?enzyme .
  ?enzyme rdf:type bp3:Protein .
  ?enzyme bp3:entityReference ?enzymeRef .
  ?enzymeRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
}
```

2.11 controls-transport-of-chemical

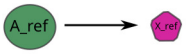
Paxtools pattern `controlsTransportOfChemical()` description

Pattern for a ProteinReference has a member PhysicalEntity that is controlling a reaction that changes cellular location of a small molecule.

Sample BioPAX structure



Inferred binary relation(s)



Meaning 2 SPARQL query

```
CONSTRUCT {
  ?enzymeRef abstraction:ControlsTransportOfChemical ?SmallMoleculeRef
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction bp3:left ?SmallMolecule1 .
  ?SmallMolecule1 rdf:type bp3:SmallMolecule .
  ?SmallMolecule1 bp3:cellularLocation ?cellularLocVocab1 .
  ?SmallMolecule1 bp3:entityReference ?SmallMoleculeRef .
  ?reaction bp3:right ?SmallMolecule2 .
  ?SmallMolecule2 rdf:type bp3:SmallMolecule .
  ?SmallMolecule2  bp3:cellularLocation ?cellularLocVocab2 .
  ?SmallMolecule2 bp3:entityReference ?SmallMoleculeRef .
  ?SmallMoleculeRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .

  ?catalysis bp3:controlled ?reaction .
  ?catalysis rdf:type/(rdfs:subClassOf*) bp3:Control .
  ?catalysis bp3:controller ?enzyme .
  ?enzyme rdf:type bp3:Protein .
  ?enzyme bp3:entityReference ?enzymeRef .
  ?enzymeRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  FILTER(?cellularLocVocab1 != ?cellularLocVocab2)
}
```

2.12 chemical-affects

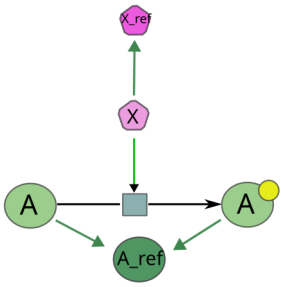
Paxtools pattern `chemicalAffectsProteinThroughBinding()` description

A small molecule is in a complex with a protein.

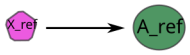
Paxtools pattern `chemicalAffectsProteinThroughControl()` description

A small molecule controls an interaction of which the protein is a participant.

Sample BioPAX structure



Inferred binary relation(s)



Meaning 2 SPARQL query

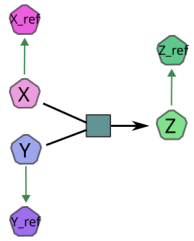
```
CONSTRUCT {
  ?chemicalCatalyzerRef abstraction:ChemicalAffects ?proteinRef .
}
WHERE {
  # the small molecule has an effect on the protein state through binding
  {
    ?complex rdf:type bp3:Complex .
    ?complex bp3:component ?proteinComponent .
    ?proteinComponent rdf:type bp3:Protein .
    # modification of the protein
    ?proteinComponent bp3:feature ?feature .
    ?feature rdf:type/(rdfs:subClassOf*) bp3:EntityFeature .
    ?complex bp3:component ?chemicalCatalyzer .
    ?chemicalCatalyzer rdf:type bp3:SmallMolecule .
    ?chemicalCatalyzer bp3:entityReference ?chemicalCatalyzerRef .
    ?chemicalCatalyzerRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
    BIND(?proteinComponent AS ?protein1)
  }
  UNION
  # the small molecule has an effect on the protein state by controlling a Conversion reaction
  {
    ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
    ?reaction bp3:left ?protLeft .
    ?protLeft rdf:type bp3:Protein .
    ?reaction bp3:right ?protRight .
    ?protRight rdf:type bp3:Protein .

    ?catalysis bp3:controlled ?reaction .
    ?catalysis rdf:type/(rdfs:subClassOf*) bp3:Control .
    ?catalysis bp3:controller ?chemicalCatalyzer .
    ?chemicalCatalyzer rdf:type bp3:SmallMolecule .
    ?chemicalCatalyzer bp3:entityReference ?chemicalCatalyzerRef .
    ?chemicalCatalyzerRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .

    # modification of the protein
    ?protRight bp3:feature ?feature .
    ?feature rdf:type/(rdfs:subClassOf*) bp3:EntityFeature .
    BIND(?protLeft AS ?protein2)
  }
  BIND(COALESCE(?protein1, ?protein2) AS ?protein)
  ?protein bp3:entityReference ?proteinRef .
  ?proteinRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
}
```

2.13 reacts-with

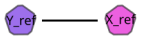
Paxtools pattern `reactsWith()` description

Constructs a pattern where first and last small molecules are substrates to the same biochemical reaction.

Sample BioPAX structure



Inferred binary relation(s)



Meaning 2 SPARQL query

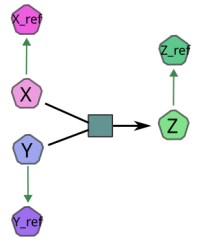
```
CONSTRUCT {
  ?smallMolecule1Ref abstraction:ReactsWith ?smallMolecule2Ref .
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:BiochemicalReaction .
  ?reaction bp3:left ?smallMolecule1 .
  ?smallMolecule1 rdf:type bp3:SmallMolecule .
  ?smallMolecule1 bp3:entityReference ?smallMolecule1Ref .
  ?smallMolecule1Ref rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  ?reaction bp3:left ?smallMolecule2 .
  ?smallMolecule2 rdf:type bp3:SmallMolecule .
  ?smallMolecule2 bp3:entityReference ?smallMolecule2Ref .
  ?smallMolecule2Ref rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  FILTER (?smallMolecule1Ref != ?smallMolecule2Ref)
}
```

2.14 used-to-produce

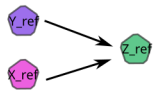
Paxtools pattern `usedToProduce()` description

Constructs a pattern where first small molecule is an input of a biochemical reaction that produces the second small molecule.

Sample BioPAX structure



Inferred binary relation(s)



Meaning 2 SPARQL query

```
CONSTRUCT {
  ?smallMolecule1Ref abstraction:UsedToProduce ?smallMolecule2Ref .
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction bp3:left ?smallMolecule1 .
  ?reaction bp3:right ?smallMolecule2 .
  ?smallMolecule1 rdf:type bp3:SmallMolecule .
  ?smallMolecule1 bp3:entityReference ?smallMolecule1Ref .
  ?smallMolecule1Ref rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  ?smallMolecule2 rdf:type bp3:SmallMolecule .
  ?smallMolecule2 bp3:entityReference ?smallMolecule2Ref .
  ?smallmolecule2Ref rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  FILTER (?smallMolecule1Ref != ?smallMolecule2Ref)
}
```

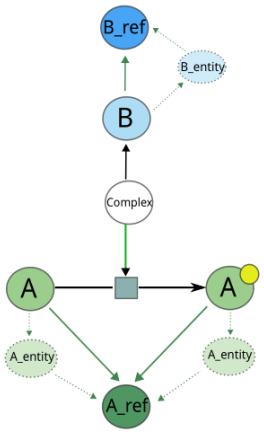
3 Meaning 3: Paxtools patterns code

3.1 controls-state-change-of

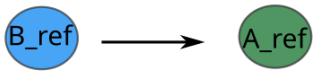
Paxtools pattern `controlsStateChange()` code

```
public static Pattern controlsStateChange()
{
    Pattern p = new Pattern(SequenceEntityReference.class, "controller-ER");
    p.add(linkedER(true), "controller-ER", "generic-controller-ER");
    p.add(erToPE(), "generic-controller-ER", "controller-simple-PE");
    p.add(linkToComplex(), "controller-simple-PE", "controller-PE");
    p.add(peToControl(), "controller-PE", "Control");
    p.add(controlToConv(), "Control", "Conversion");
    p.add(new NOT(participantER()), "Conversion", "controller-ER");
    p.add(new Participant(RelType.INPUT, true), "Control", "Conversion", "input-PE");

    p.add(new NOT(new ConversionSide(ConversionSide.Type.OTHER_SIDE)), "input-PE",
    "Conversion", "input-PE");

    p.add(linkToSpecific(), "input-PE", "input-simple-PE");
    p.add(new Type(SequenceEntity.class), "input-simple-PE");
    p.add(peToER(), "input-simple-PE", "changed-generic-ER");
    p.add(new ConversionSide(ConversionSide.Type.OTHER_SIDE), "input-PE",
    "Conversion", "output-PE");

    p.add(new NOT(new ConversionSide(ConversionSide.Type.OTHER_SIDE)),
    "output-PE", "Conversion", "output-PE");

    p.add(equal(false), "input-PE", "output-PE");
    p.add(linkToSpecific(), "output-PE", "output-simple-PE");
    p.add(peToER(), "output-simple-PE", "changed-generic-ER");
    p.add(linkedER(false), "changed-generic-ER", "changed-ER");

    return p;
}
```

Sample BioPAX structure



Inferred binary relation(s)



Meaning 3 SPARQL query

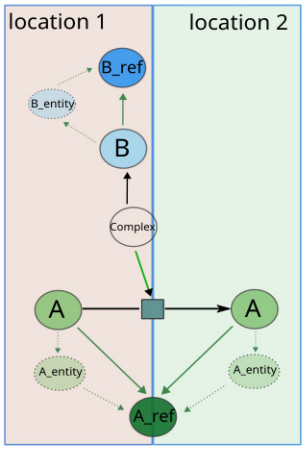
```
CONSTRUCT {
  ?enzymeRef abstraction:ControlsStateChangeOf ?protRef
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?catalysis bp3:controlled ?reaction .
  ?catalysis (rdf:type/rdfs:subClassOf*) bp3:Control .
  ?catalysis bp3:controller/((bp3:component|bp3:memberPhysicalEntity)*) ?enzyme .
  ?enzyme rdf:type bp3:Protein .
  ?enzyme bp3:entityReference ?enzymeRef .
  ?enzymeRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .

  ?reaction bp3:left/(bp3:memberPhysicalEntity)* ?protLeft .
  ?protLeft rdf:type bp3:Protein .
  ?protLeft bp3:entityReference ?protRef .

  ?reaction bp3:right/(bp3:memberPhysicalEntity)* ?protRight .
  ?protRight rdf:type bp3:Protein .
  ?protRight bp3:entityReference ?protRef .
  ?protRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .

  # State change of the product
  ?protRight bp3:feature ?feature .
  ?feature rdf:type/(rdfs:subClassOf*) bp3:EntityFeature .
  FILTER (?enzymeRef != ?protRef)
  FILTER (?protLeft != ?protRight)
}
```

3.2 controls-transport-of

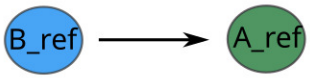
Paxtools pattern `controlsTransport()` code

```
public static Pattern controlsTransport()
{
    Pattern p = controlsStateChange();
    p.add(new OR(
        new MappedConst(hasDifferentCompartments(), 0, 1),
        new MappedConst(hasDifferentCompartments(), 2, 3)),
        "input-simple-PE", "output-simple-PE", "input-PE", "output-PE");

    return p;
}
```

Sample BioPAX structure



Inferred binary relation(s)



Meaning 3 SPARQL query

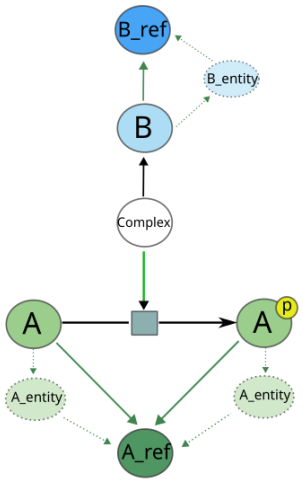
```
CONSTRUCT {
  ?enzymeRef abstraction:ControlsTransportOf ?protRef
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction bp3:left/(bp3:memberPhysicalEntity)* ?protLeft .
  ?protLeft rdf:type bp3:Protein .
  ?protLeft bp3:entityReference ?protRef .

  ?reaction bp3:right/(bp3:memberPhysicalEntity)* ?protRight .
  ?protRight rdf:type bp3:Protein .
  ?protRight bp3:entityReference ?protRef .
  ?protRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  # Define cellular locations for Protein 1 and Protein 2
  ?protLeft bp3:cellularLocation ?cellularLocVocab1 .
  ?protRight bp3:cellularLocation ?cellularLocVocab2 .

  ?catalysis bp3:controlled ?reaction .
  ?catalysis (rdf:type/rdfs:subClassOf*) bp3:Control .
  ?catalysis bp3:controller/((bp3:component|bp3:memberPhysicalEntity)*) ?enzyme .
  ?enzyme rdf:type bp3:Protein .
  ?enzyme bp3:entityReference ?enzymeRef .
  ?enzymeRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  FILTER (?cellularLocVocab1 != ?cellularLocVocab2)
  FILTER (?enzymeRef != ?protRef)
  FILTER (?protLeft != ?protRight)
}
```

3.3 controls-phosphorylation-of

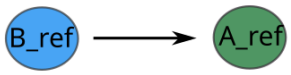
Paxtools pattern `controlsPhosphorylation()` code

```
public static Pattern controlsPhosphorylation()
{
    Pattern p = controlsStateChange();
    p.add(new NOT(ConBox.linkToSpecific()), "input-PE", "output-simple-PE");
    p.add(new NOT(ConBox.linkToSpecific()), "output-PE", "input-simple-PE");
    p.add(new ModificationChangeConstraint(ModificationChangeConstraint.Type.ANY,
        "phospho"), "input-simple-PE", "output-simple-PE");
    return p;
}
```

Sample BioPAX structure



Inferred binary relation(s)



Meaning 3 SPARQL query

```
CONSTRUCT {
?enzymeRef abstraction:ControlsPhosphorylationOf ?protRef
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?catalysis bp3:controlled ?reaction .
  ?catalysis rdf:type/(rdfs:subClassOf*) bp3:Control .
  ?catalysis bp3:controller/((bp3:component|bp3:memberPhysicalEntity)*) ?enzyme .
  ?enzyme rdf:type bp3:Protein .
  ?enzyme bp3:entityReference ?enzymeRef .
  ?enzymeRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .

  ?reaction bp3:left/(bp3:memberPhysicalEntity)* ?protLeft .
  ?protLeft rdf:type bp3:Protein .
  ?protLeft bp3:entityReference ?protRef .

  ?reaction bp3:right/(bp3:memberPhysicalEntity)* ?protRight .
  ?protRight rdf:type bp3:Protein .
  ?protRight bp3:entityReference ?protRef .
  ?protRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  # Specify that ?protRight is phosphorylated
  ?protRight bp3:feature ?feature .
  ?feature rdf:type bp3:ModificationFeature .
  ?feature bp3:modificationType ?modificationType .
  ?modificationType rdf:type bp3:SequenceModificationVocabulary .
  ?modificationType bp3:term ?modificationTerm .
  FILTER(CONTAINS(LCASE(?modificationTerm), "phospho"))
  FILTER(?enzymeRef != ?protRef)
  FILTER(?protLeft != ?protRight)
}
```

3.4 controls-expression-of

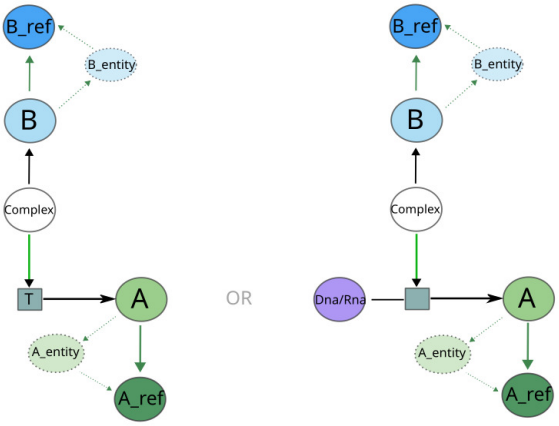
Paxtools pattern `controlsExpressionWithTemplateReac()` code

```
public static Pattern controlsExpressionWithTemplateReac()
{
    Pattern p = new Pattern(SequenceEntityReference.class, "TF-ER");
    p.add(linkedER(true), "TF-ER", "TF-generic-ER");
    p.add(erToPE(), "TF-generic-ER", "TF-SPE");
    p.add(linkToComplex(), "TF-SPE", "TF-PE");
    p.add(peToControl(), "TF-PE", "Control");
    p.add(controlToTempReac(), "Control", "TempReac");
    p.add(product(), "TempReac", "product-PE");
    p.add(linkToSpecific(), "product-PE", "product-SPE");
    p.add(new Type(SequenceEntity.class), "product-SPE");
    p.add(peToER(), "product-SPE", "product-generic-ER");
    p.add(linkedER(false), "product-generic-ER", "product-ER");
    p.add(equal(false), "TF-ER", "product-ER");
    return p;
}
```

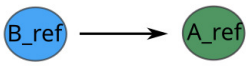
Paxtools pattern `controlsExpressionWithConversion()` code

```
public static Pattern controlsExpressionWithConversion()
{
    Pattern p = new Pattern(SequenceEntityReference.class, "TF-ER");
    p.add(linkedER(true), "TF-ER", "TF-generic-ER");
    p.add(erToPE(), "TF-generic-ER", "TF-SPE");
    p.add(linkToComplex(), "TF-SPE", "TF-PE");
    p.add(peToControl(), "TF-PE", "Control");
    p.add(controlToConv(), "Control", "Conversion");
    p.add(new Size(right(), 1, Size.Type.EQUAL), "Conversion");
    p.add(new OR(new MappedConst(new Empty(left()), 0),
    new MappedConst(new ConstraintAdapter(1)
    {
        @Override
        public boolean satisfies(Match match, int... ind)
        {
            Conversion cnv = (Conversion) match.get(ind[0]);
            Set<PhysicalEntity> left = cnv.getLeft();
            if (left.size() > 1) return false;
            if (left.isEmpty()) return true;
            PhysicalEntity pe = left.iterator().next();
            if (pe instanceof NucleicAcid)
            {
                PhysicalEntity rPE = cnv.getRight().iterator().next();
                return rPE instanceof Protein;
            }
            return false;
        }
    }, 0)), "Conversion");
    p.add(right(), "Conversion", "right-PE");
    p.add(linkToSpecific(), "right-PE", "right-SPE");
    p.add(new Type(SequenceEntity.class), "right-SPE");
    p.add(peToER(), "right-SPE", "product-generic-ER");
    p.add(linkedER(false), "product-generic-ER", "product-ER");
    p.add(equal(false), "TF-ER", "product-ER");
    return p;
}
```

Sample BioPAX structure



Inferred binary relation(s)



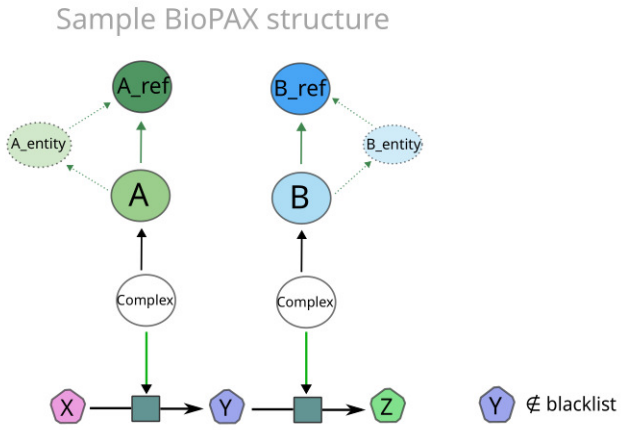
Meaning 3 SPARQL query

```
CONSTRUCT {
  ?enzymeRef abstraction:ControlsExpressionOf ?productRef
}
WHERE {
  # Case where the reaction is a Conversion
  {
    VALUES ?NucleicAcid { bp3:Dna bp3:Rna }
    ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
    ?reaction bp3:left/(bp3:memberPhysicalEntity)* ?left .
    ?left rdf:type ?NucleicAcid .
    ?reaction bp3:right/(bp3:memberPhysicalEntity)* ?product .
    ?product rdf:type bp3:Protein .
    ?product bp3:entityReference ?productRef .
    ?productRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
    ?catalysis bp3:controlled ?reaction .
    ?catalysis rdf:type/(rdfs:subClassOf*) bp3:Control .
    ?catalysis bp3:controller/((bp3:component|bp3:memberPhysicalEntity)*) ?enzyme .
    ?enzyme rdf:type bp3:Protein .
    ?enzyme bp3:entityReference ?enzymeRef .
    ?enzymeRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  }
  UNION
  # Case where the reaction is a TemplateReaction
  {
    ?reaction rdf:type/(rdfs:subClassOf*) bp3:TemplateReaction .
    ?reaction bp3:product/(bp3:memberPhysicalEntity)* ?product .
    ?product rdf:type bp3:Protein .
    ?product bp3:entityReference ?productRef .
    ?productRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
    ?catalysis bp3:controlled ?reaction .
    ?catalysis rdf:type bp3:TemplateReactionRegulation .
    ?catalysis bp3:controller/((bp3:component|bp3:memberPhysicalEntity)*) ?enzyme .
    ?enzyme rdf:type bp3:Protein .
    ?enzyme bp3:entityReference ?enzymeRef .
    ?enzymeRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  }
  FILTER (?enzymeRef != ?productRef)
}
```

3.5 catalysis-precedes

Paxtools pattern `catalysisPrecedes()` code

```
public static Pattern catalysisPrecedes(Blacklist blacklist)
{
    Pattern p = new Pattern(SequenceEntityReference.class, "first-ER");
    p.add(linkedER(true), "first-ER", "first-generic-ER");
    p.add(erToPE(), "first-generic-ER", "first-simple-controller-PE");
    p.add(linkToComplex(), "first-simple-controller-PE", "first-controller-PE");
    p.add(peToControl(), "first-controller-PE", "first-Control");
    p.add(controlToConv(), "first-Control", "first-Conversion");
    p.add(new Participant(RelType.OUTPUT, blacklist, true),
    "first-Control", "first-Conversion", "linker-PE");
    p.add(new NOT(new ConstraintChain(new ConversionSide(ConversionSide.Type.OTHER_SIDE),
    linkToSpecific())), "linker-PE", "first-Conversion", "linker-PE");
    p.add(type(SmallMolecule.class), "linker-PE");
    p.add(new ParticipatesInConv(RelType.INPUT, blacklist),
    "linker-PE", "second-Conversion");
    p.add(new NOT(new ConstraintChain(new ConversionSide(ConversionSide.Type.OTHER_SIDE),
    linkToSpecific())), "linker-PE",
    "second-Conversion", "linker-PE");
    p.add(equal(false), "first-Conversion", "second-Conversion");
    // make sure that conversions are not replicates or reverse of each other
    // and also outward facing sides of reactions contain at least one non ubique
    p.add(new ConstraintAdapter(3, blacklist)
    {
        @Override
        public boolean satisfies(Match match, int... ind)
        {
            Conversion cnv1 = (Conversion) match.get(ind[0]);
            Conversion cnv2 = (Conversion) match.get(ind[1]);
            SmallMolecule linker = (SmallMolecule) match.get(ind[2]);
            Set<PhysicalEntity> input1 = cnv1.getLeft().contains(linker) ? cnv1.getRight() :
            cnv1.getLeft();
            Set<PhysicalEntity> input2 = cnv2.getLeft().contains(linker) ? cnv2.getLeft() :
            cnv2.getRight();
            Set<PhysicalEntity> output1 = cnv1.getLeft().contains(linker) ? cnv1.getLeft() :
            cnv1.getRight();
            Set<PhysicalEntity> output2 = cnv2.getLeft().contains(linker) ? cnv2.getRight() :
            cnv2.getLeft();
            if (input1.equals(input2) && output1.equals(output2)) return false;
            if (input1.equals(output2) && output1.equals(input2)) return false;
            if (blacklist != null)
            {
                Set<PhysicalEntity> set = new HashSet<>(input1);
                set = blacklist.getNonUbiques(set, RelType.INPUT);
                set.removeAll(output2);
                if (set.isEmpty())
                {
                    set.addAll(output2);
                    set = blacklist.getNonUbiques(set, RelType.OUTPUT);
                    set.removeAll(input1);

                    if (set.isEmpty()) return false;
                }
            }
            return true;
        }
    }, "first-Conversion", "second-Conversion", "linker-PE");
    p.add(new RelatedControl(RelType.INPUT, blacklist), "linker-PE", "second-Conversion",
    "second-Control");
    p.add(controllerPE(), "second-Control", "second-controller-PE");
    p.add(new NOT(compToER()), "second-controller-PE", "first-ER");
    p.add(linkToSpecific(), "second-controller-PE", "second-simple-controller-PE");
    p.add(type(SequenceEntity.class), "second-simple-controller-PE");
    p.add(peToER(), "second-simple-controller-PE", "second-generic-ER");
    p.add(linkedER(false), "second-generic-ER", "second-ER");
    p.add(equal(false), "first-ER", "second-ER");
    return p;
}
```


Sample BioPAX structure
A_ref
B_ref
A_entity
B_entity
A
B
Complex
Complex
X
Y
Z
Y ∉ blacklist
Inferred binary relation(s)
A_ref
B_ref

Meaning 3 SPARQL query

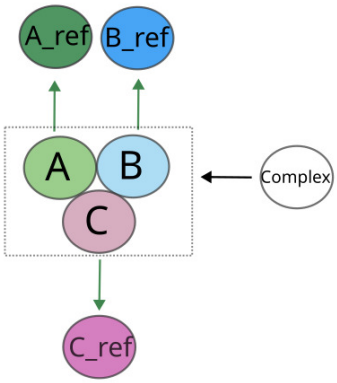
```
CONSTRUCT {
  ?enzymeRefA abstraction:CatalysisPrecedes ?enzymeRefB
  }
WHERE {
  ?reaction1 rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction1 bp3:left ?leftMolecule1 .
  ?leftMolecule1 rdf:type bp3:SmallMolecule .
  # get the connecting molecule
  ?reaction1 bp3:right ?connectingMolecule .
  ?connectingMolecule rdf:type bp3:SmallMolecule .
  ?connectingMolecule bp3:entityReference ?connectingMoleculeRef .
  ?connectingMoleculeRef bp3:xref ?connectingMoleculeXref .
  ?connectingMoleculeXref rdf:type bp3:UnificationXref .
  ?connectingMoleculeXref bp3:db "ChEBI" .
  ?connectingMoleculeXref bp3:id ?connectingMoleculeID .
  FILTER (?connectingMoleculeID NOT IN ("CHEBI:15377", "CHEBI:24636", "CHEBI:15378", "CHEBI:15379",
  "CHEBI:57783","CHEBI:13392", "CHEBI:16474", "CHEBI:58349", "CHEBI:18009", "CHEBI:13390","CHEBI:25523",
  "CHEBI:25524", "CHEBI:30616", "CHEBI:15422", "CHEBI:16761","CHEBI:456216", "CHEBI:57945", "CHEBI:16908",
  "CHEBI:57540", "CHEBI:15846","CHEBI:16526", "CHEBI:29888", "CHEBI:35782", "CHEBI:68836", "CHEBI:18361",
    "CHEBI:43474", "CHEBI:35780", "CHEBI:18367", "CHEBI:26078", "CHEBI:77740",
    "CHEBI:28931", "CHEBI:58307", "CHEBI:17877", "CHEBI:17552", "CHEBI:58189",
    "CHEBI:37565", "CHEBI:15996"))
  ?reaction2 bp3:left ?connectingMolecule .
  ?reaction2 rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction2 bp3:right ?rightMolecule2 .
  ?rightMolecule2 rdf:type bp3:SmallMolecule .

  # Get catalysis of the first reaction
  ?catalysis1 bp3:controlled ?reaction1 .
  ?catalysis1 (rdf:type/rdfs:subClassOf*) bp3:Control .
  ?catalysis1 bp3:controller/((bp3:component|bp3:memberPhysicalEntity)*) ?enzymeProtA .
  ?enzymeProtA rdf:type bp3:Protein .
  ?enzymeProtA bp3:entityReference ?enzymeRefA .
  ?enzymeRefA rdf:type/(rdfs:subClassOf*) bp3:EntityReference .

  # get catalysis of the second reaction
  ?catalysis2 bp3:controlled ?reaction2 .
  ?catalysis2 (rdf:type/rdfs:subClassOf*) bp3:Control .
  ?catalysis2 bp3:controller/((bp3:component|bp3:memberPhysicalEntity)*) ?enzymeProtB .
  ?enzymeProtB rdf:type bp3:Protein .
  ?enzymeProtB bp3:entityReference ?enzymeRefB .
  ?enzymeRefB rdf:type/(rdfs:subClassOf*) bp3:EntityReference .

  FILTER (?enzymeRefA != ?enzymeRefB)
  FILTER (?leftMolecule1 != ?rightMolecule2)
  FILTER (?leftMolecule1 != ?connectingMolecule)
  FILTER (?rightMolecule2 != ?connectingMolecule)
}
```

3.6 in-complex-with

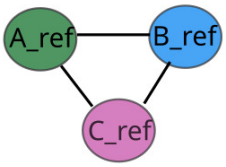
Paxtools pattern `inComplexWith()` code

```
public static Pattern inComplexWith()
{
    Pattern p = new Pattern(SequenceEntityReference.class, "Protein-1");
    p.add(linkedER(true), "Protein-1", "generic-Protein-1");
    p.add(erToPE(), "generic-Protein-1", "SPE1");
    p.add(linkToComplex(), "SPE1", "PE1");
    p.add(new PathConstraint("PhysicalEntity/componentOf"), "PE1", "Complex");
    p.add(new PathConstraint("Complex/component"), "Complex", "PE2");
    p.add(equal(false), "PE1", "PE2");
    p.add(linkToSpecific(), "PE2", "SPE2");
    p.add(peToER(), "SPE2", "generic-Protein-2");
    p.add(linkedER(false), "generic-Protein-2", "Protein-2");
    p.add(equal(false), "Protein-1", "Protein-2");
    p.add(new Type(SequenceEntityReference.class), "Protein-2");
    return p;
}
```

Sample BioPAX structure



Inferred binary relation(s)



Meaning 3 SPARQL query

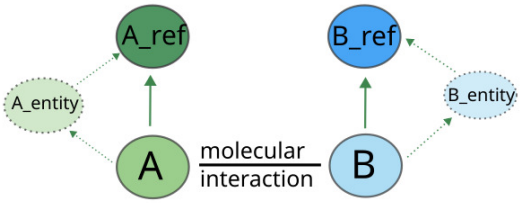
```
CONSTRUCT {
  ?prot1Ref abstraction:InComplexWith ?prot2Ref
}
WHERE {
  # Case 1: Complex with several different proteins
  {
    ?complex rdf:type/(rdfs:subClassOf*) bp3:Complex .
    ?complex (bp3:component|bp3:memberPhysicalEntity)* ?prot1 .
    ?complex (bp3:component|bp3:memberPhysicalEntity)* ?prot2 .
    ?prot1 rdf:type bp3:Protein .
    ?prot2 rdf:type bp3:Protein .
    ?prot1 bp3:entityReference ?prot1Ref .
    ?prot2 bp3:entityReference ?prot2Ref .
    ?prot1Ref rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
    ?prot2Ref rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
    # Avoid self-pairing and duplicate pairs
    FILTER (STR(?prot1Ref) < STR(?prot2Ref))
  }
  UNION
  # Case 2: Same protein with stoichiometry > 1
  {
    ?complex rdf:type/(rdfs:subClassOf*) bp3:Complex .
    # Check for stoichiometry > 1
    ?complex bp3:componentStoichiometry ?stoich .
    ?stoich bp3:physicalEntity ?entity1 .
    ?stoich bp3:stoichiometricCoefficient ?coeff .
    FILTER (?coeff > 1)
    ?complex (bp3:component|bp3:memberPhysicalEntity)* ?prot1 .
    ?prot1 rdf:type bp3:Protein .
    ?prot1 bp3:entityReference ?prot1Ref .
    ?prot1Ref rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
    # For self-pairing, use the same entity
    BIND (?prot1Ref AS ?prot2Ref)
  }
  FILTER (?prot1Ref != ?prot2Ref)
}
```

3.7 interacts-with

Paxtools pattern `molecularInteraction()` code

```
public static Pattern molecularInteraction()
{
    Pattern p = new Pattern(SequenceEntityReference.class, "Protein-1");
    p.add(linkedER(true), "Protein-1", "generic-Protein-1");
    p.add(erToPE(), "generic-Protein-1", "SPE1");
    p.add(linkToComplex(), "SPE1", "PE1");
    p.add(new PathConstraint("PhysicalEntity/participantOf:MolecularInteraction"),
    "PE1", "MI");
    p.add(participant(), "MI", "PE2");
    p.add(equal(false), "PE1", "PE2");

    // participants are not both baits or preys
    p.add(new NOT(new AND(new MappedConst(isPrey(), 0), new MappedConst(isPrey(), 1))),
    "PE1", "PE2");
    p.add(new NOT(new AND(new MappedConst(isBait(), 0), new MappedConst(isBait(), 1))),
    "PE1", "PE2");

    p.add(linkToSpecific(), "PE2", "SPE2");
    p.add(type(SequenceEntity.class), "SPE2");
    p.add(new PEChainsIntersect(false), "SPE1", "PE1", "SPE2", "PE2");
    p.add(peToER(), "SPE2", "generic-Protein-2");
    p.add(linkedER(false), "generic-Protein-2", "Protein-2");
    p.add(equal(false), "Protein-1", "Protein-2");
    return p;
}
```

Sample BioPAX structure



Inferred binary relation(s)

Meaning 3 SPARQL query

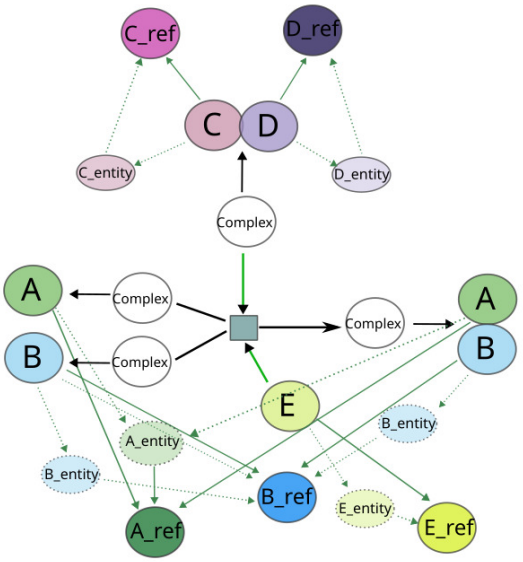
```
CONSTRUCT {
  ?participant1Ref abstraction:InteractsWith ?participant2Ref
}
WHERE {
  VALUES ?participantType { bp3:Protein bp3:SmallMolecule }
  ?MolecularInteraction rdf:type/(rdfs:subClassOf*) bp3:MolecularInteraction .
  ?MolecularInteraction bp3:participant/(bp3:memberPhysicalEntity)* ?participant1 .
  ?participant1 rdf:type ?participantType .
  ?participant1 bp3:entityReference ?participant1Ref .
  ?participant1Ref rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  ?MolecularInteraction bp3:participant/(bp3:memberPhysicalEntity)* ?participant2 .
  ?participant2 rdf:type ?participantType .
  ?participant2 bp3:entityReference ?participant2Ref .
  ?participant2Ref rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  FILTER (STR(?participant1Ref) < STR(?participant2Ref))
  FILTER (?participant1Ref != ?participant2Ref)
}
```

3.8 neighbor-of

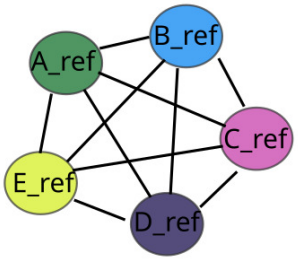
Paxtools pattern `neighborOf()` code

```
public static Pattern neighborOf()
{
    Pattern p = new Pattern(SequenceEntityReference.class, "Protein 1");
    p.add(linkedER(true), "Protein 1", "generic Protein 1");
    p.add(erToPE(), "generic Protein 1", "SPE1");
    p.add(linkToComplex(), "SPE1", "PE1");
    p.add(peToInter(), "PE1", "Inter");
    p.add(interToPE(), "Inter", "PE2");
    p.add(linkToSpecific(), "PE2", "SPE2");
    p.add(equal(false), "SPE1", "SPE2");
    p.add(type(SequenceEntity.class), "SPE2");
    p.add(peToER(), "SPE2", "generic Protein 2");
    p.add(linkedER(false), "generic Protein 2", "Protein 2");
    p.add(equal(false), "Protein 1", "Protein 2");
    return p;
}
```

Sample BioPAX structure



Inferred binary relation(s)



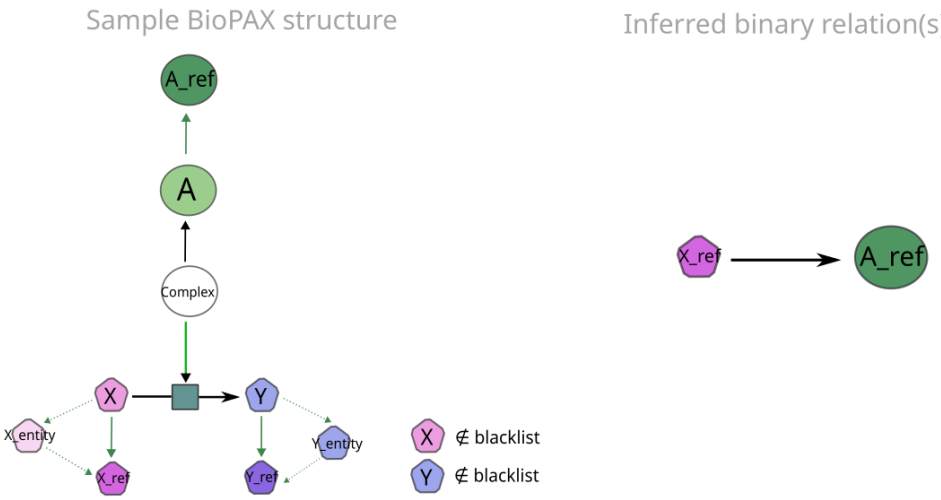
Meaning 3 SPARQL query

```
CONSTRUCT {
  ?participant1Ref abstraction:NeighborOf ?participant2Ref
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Interaction .

  ?reaction ((^bp3:controlled/bp3:controller)|bp3:left|bp3:right) ?participant1 .
  ?participant1 (bp3:component|bp3:memberPhysicalEntity)* ?participant1Prot .
  ?participant1Prot rdf:type bp3:Protein .
  ?participant1Prot bp3:entityReference ?participant1Ref .
  ?participant1Ref rdf:type/(rdfs:subClassOf*) bp3:EntityReference .

  ?reaction ((^bp3:controlled/bp3:controller)|bp3:left|bp3:right) ?participant2 .
  ?participant2 (bp3:component|bp3:memberPhysicalEntity)* ?participant2Prot .
  ?participant2Prot rdf:type bp3:Protein .
  ?participant2Prot bp3:entityReference ?participant2Ref .
  ?participant2Ref rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  FILTER (STR(?participant1Ref) < STR(?participant2Ref))
  FILTER (?participant1Ref != ?participant2Ref)
}
```

3.9 consumption-controlled-by

Paxtools pattern `controlsMetabolicCatalysis()` code

```
public static Pattern controlsMetabolicCatalysis(Blacklist blacklist, boolean consumption)
{
    Pattern p = new Pattern(SequenceEntityReference.class, "controller-ER");
    p.add(linkedER(true), "controller-ER", "controller-generic-ER");
    p.add(erToPE(), "controller-generic-ER", "controller-simple-PE");
    p.add(linkToComplex(), "controller-simple-PE", "controller-PE");
    p.add(peToControl(), "controller-PE", "Control");
    p.add(controlToConv(), "Control", "Conversion");
    p.add(new NOT(participantER()), "Conversion", "controller-ER");

    p.add(new Participant(consumption ? RelType.INPUT : RelType.OUTPUT, blacklist, true),
        "Control", "Conversion", "part-PE");

    p.add(linkToSimple(blacklist), "part-PE", "part-SM");
    p.add(notGeneric(), "part-SM");
    p.add(type(SmallMolecule.class), "part-SM");
    p.add(peToER(), "part-SM", "part-SMR");

    // The small molecule is associated only with left or right, but not both.
    p.add(new XOR(
        new MappedConst(new InterToPartER(InterToPartER.Direction.LEFT), 0, 1),
        new MappedConst(new InterToPartER(InterToPartER.Direction.RIGHT), 0, 1)),
        "Conversion", "part-SMR");

    return p;
}
```



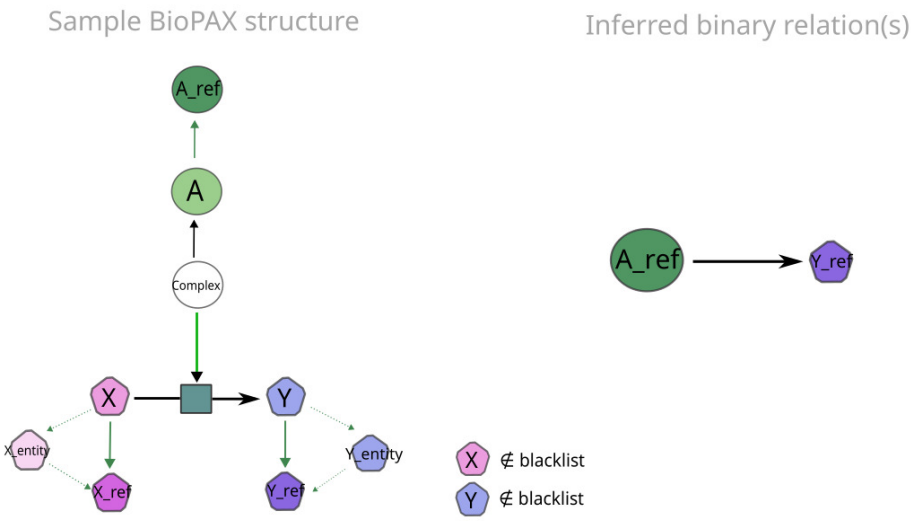
Meaning 3 SPARQL query

```
CONSTRUCT {
  ?reactantRef abstraction:ConsumptionControlledBy ?enzymeRef
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction bp3:left/(bp3:memberPhysicalEntity)* ?reactant .
  ?reactant rdf:type bp3:SmallMolecule .
  ?reactant bp3:entityReference ?reactantRef .
  ?reactantRef rdf:type bp3:SmallMoleculeReference .
  ?reactantRef bp3:xref ?reactantRefXref .
  ?reactantRefXref rdf:type bp3:UnificationXref .
  ?reactantRefXref bp3:db "ChEBI" .
  ?reactantRefXref bp3:id ?reactantID .

  FILTER (?reactantID NOT IN ("CHEBI:15377", "CHEBI:24636", "CHEBI:15378", "CHEBI:15379", "CHEBI:57783",
    "CHEBI:13392", "CHEBI:16474", "CHEBI:58349", "CHEBI:18009", "CHEBI:13390",
    "CHEBI:25523", "CHEBI:25524", "CHEBI:30616", "CHEBI:15422", "CHEBI:16761",
    "CHEBI:456216", "CHEBI:57945", "CHEBI:16908", "CHEBI:57540", "CHEBI:15846",
    "CHEBI:16526", "CHEBI:29888", "CHEBI:35782", "CHEBI:68836", "CHEBI:18361",
    "CHEBI:43474", "CHEBI:35780", "CHEBI:18367", "CHEBI:26078", "CHEBI:77740",
    "CHEBI:28931", "CHEBI:58307", "CHEBI:17877", "CHEBI:17552", "CHEBI:58189",
    "CHEBI:37565", "CHEBI:15996"))

  ?reaction bp3:right/(bp3:memberPhysicalEntity)* ?product .
  ?product rdf:type bp3:SmallMolecule .
  ?product bp3:entityReference ?productRef .
  ?productRef rdf:type bp3:SmallMoleculeReference .
  ?productRef bp3:xref ?productRefXref .
  ?productRefXref rdf:type bp3:UnificationXref .
  ?productRefXref bp3:db "ChEBI" .
  ?productRefXref bp3:id ?productID .

  FILTER (?productID NOT IN ("CHEBI:15377", "CHEBI:24636", "CHEBI:15378", "CHEBI:15379", "CHEBI:57783",
    "CHEBI:13392", "CHEBI:16474", "CHEBI:58349", "CHEBI:18009", "CHEBI:13390",
    "CHEBI:25523", "CHEBI:25524", "CHEBI:30616", "CHEBI:15422", "CHEBI:16761",
    "CHEBI:456216", "CHEBI:57945", "CHEBI:16908", "CHEBI:57540", "CHEBI:15846",
    "CHEBI:16526", "CHEBI:29888", "CHEBI:35782", "CHEBI:68836", "CHEBI:18361",
    "CHEBI:43474", "CHEBI:35780", "CHEBI:18367", "CHEBI:26078", "CHEBI:77740",
    "CHEBI:28931", "CHEBI:58307", "CHEBI:17877", "CHEBI:17552", "CHEBI:58189",
    "CHEBI:37565", "CHEBI:15996"))

  ?catalysis bp3:controlled ?reaction .
  ?catalysis (rdf:type/rdfs:subClassOf*) bp3:Control .
  ?catalysis bp3:controller/((bp3:component|bp3:memberPhysicalEntity)*) ?enzyme .
  ?enzyme rdf:type bp3:Protein .
  ?enzyme bp3:entityReference ?enzymeRef .
  ?enzymeRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  FILTER (?reactantRef != ?enzymeRef)
  FILTER (?reactant != ?product)
}
```

3.10 controls-production-of

Paxtools pattern code

There is no Paxtools Pattern for the SIF rule `controls-production-of`. Since this rule is based on the same pattern than the SIF rule `consumption-controlled-by`, we used the same logic for the meaning 3.



Meaning 3 SPARQL query

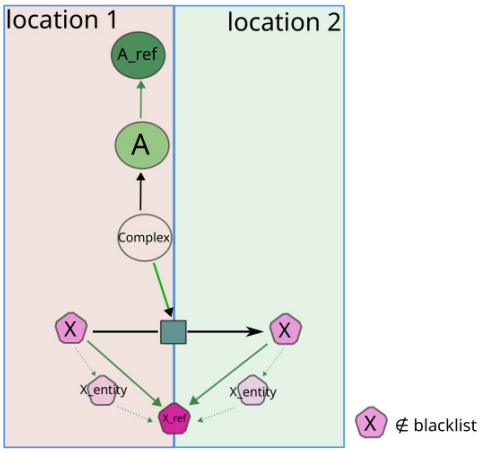
```
CONSTRUCT {
  ?enzymeRef abstraction:ControlsProductionOf ?productRef
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction bp3:left/(bp3:memberPhysicalEntity)* ?reactant .
  ?reactant rdf:type bp3:SmallMolecule .
  ?reactant bp3:entityReference ?reactantRef .
  ?reactantRef rdf:type bp3:SmallMoleculeReference .
  ?reactantRef bp3:xref ?reactantRefXref .
  ?reactantRefXref rdf:type bp3:UnificationXref .
  ?reactantRefXref bp3:db "ChEBI" .
  ?reactantRefXref bp3:id ?reactantID .

  FILTER (?reactantID NOT IN ("CHEBI:15377", "CHEBI:24636", "CHEBI:15378", "CHEBI:15379", "CHEBI:57783",
    "CHEBI:13392", "CHEBI:16474", "CHEBI:58349", "CHEBI:18009", "CHEBI:13390",
    "CHEBI:25523", "CHEBI:25524", "CHEBI:30616", "CHEBI:15422", "CHEBI:16761",
    "CHEBI:456216", "CHEBI:57945", "CHEBI:16908", "CHEBI:57540", "CHEBI:15846",
    "CHEBI:16526", "CHEBI:29888", "CHEBI:35782", "CHEBI:68836", "CHEBI:18361",
    "CHEBI:43474", "CHEBI:35780", "CHEBI:18367", "CHEBI:26078", "CHEBI:77740",
    "CHEBI:28931", "CHEBI:58307", "CHEBI:17877", "CHEBI:17552", "CHEBI:58189",
    "CHEBI:37565", "CHEBI:15996"))

  ?reaction bp3:right/(bp3:memberPhysicalEntity)* ?product .
  ?product rdf:type bp3:SmallMolecule .
  ?product bp3:entityReference ?productRef .
  ?productRef rdf:type bp3:SmallMoleculeReference .
  ?productRef bp3:xref ?productRefXref .
  ?productRefXref rdf:type bp3:UnificationXref .
  ?productRefXref bp3:db "ChEBI" .
  ?productRefXref bp3:id ?productID .

  FILTER (?productID NOT IN ("CHEBI:15377", "CHEBI:24636", "CHEBI:15378", "CHEBI:15379", "CHEBI:57783",
    "CHEBI:13392", "CHEBI:16474", "CHEBI:58349", "CHEBI:18009", "CHEBI:13390",
    "CHEBI:25523", "CHEBI:25524", "CHEBI:30616", "CHEBI:15422", "CHEBI:16761",
    "CHEBI:456216", "CHEBI:57945", "CHEBI:16908", "CHEBI:57540", "CHEBI:15846",
    "CHEBI:16526", "CHEBI:29888", "CHEBI:35782", "CHEBI:68836", "CHEBI:18361",
    "CHEBI:43474", "CHEBI:35780", "CHEBI:18367", "CHEBI:26078", "CHEBI:77740",
    "CHEBI:28931", "CHEBI:58307", "CHEBI:17877", "CHEBI:17552", "CHEBI:58189",
    "CHEBI:37565", "CHEBI:15996"))

  ?catalysis bp3:controlled ?reaction .
  ?catalysis (rdf:type/rdfs:subClassOf*) bp3:Control .
  ?catalysis bp3:controller/((bp3:component|bp3:memberPhysicalEntity)*) ?enzyme .
  ?enzyme rdf:type bp3:Protein .
  ?enzyme bp3:entityReference ?enzymeRef .
  ?enzymeRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  FILTER(?enzymeRef != ?productRef)
  FILTER(?reactant != ?product)
}
```

3.11 controls-transport-of-chemical

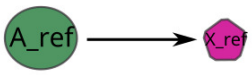
Paxtools pattern `controlsTransportOfChemical()` code

```
public static Pattern controlsTransportOfChemical(Blacklist blacklist)
{
    Pattern p = new Pattern(SequenceEntityReference.class, "controller-ER");
    p.add(linkedER(true), "controller-ER", "controller-generic-ER");
    p.add(erToPE(), "controller-generic-ER", "controller-simple-PE");
    p.add(linkToComplex(), "controller-simple-PE", "controller-PE");
    p.add(peToControl(), "controller-PE", "Control");
    p.add(controlToConv(), "Control", "Conversion");
    p.add(new Participant(RelType.INPUT, blacklist, true), "Control", "Conversion",
    "input-PE");
    p.add(linkToSimple(blacklist), "input-PE", "input-simple-PE");
    p.add(new Type(SmallMolecule.class), "input-simple-PE");
    p.add(notGeneric(), "input-simple-PE");
    p.add(peToER(), "input-simple-PE", "changed-generic-SMR");
    p.add(new ConversionSide(ConversionSide.Type.OTHER_SIDE, blacklist, RelType.OUTPUT),
    "input-PE", "Conversion", "output-PE");
    p.add(equal(false), "input-PE", "output-PE");
    p.add(linkToSimple(blacklist), "output-PE", "output-simple-PE");
    p.add(new Type(SmallMolecule.class), "output-simple-PE");
    p.add(notGeneric(), "output-simple-PE");
    p.add(peToER(), "output-simple-PE", "changed-generic-SMR");
    p.add(linkedER(false), "changed-generic-SMR", "changed-SMR");
    p.add(new OR(
        new MappedConst(hasDifferentCompartments(), 0, 1),
        new MappedConst(hasDifferentCompartments(), 2, 3)),
        "input-simple-PE", "output-simple-PE", "input-PE", "output-PE");

    return p;
}
```

Sample BioPAX structure



Inferred binary relation(s)



Meaning 3 SPARQL query

```
CONSTRUCT {
  ?enzymeRef abstraction:ControlsTransportOfChemical ?SmallMoleculeRef
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction bp3:left/(bp3:memberPhysicalEntity)* ?SmallMolecule1 .
  ?SmallMolecule1 rdf:type bp3:SmallMolecule .
  ?SmallMolecule1 bp3:cellularLocation ?cellularLocVocab1 .
  ?SmallMolecule1 bp3:entityReference ?SmallMoleculeRef .
  ?SmallMoleculeRef bp3:xref ?SmallMoleculeRefXref .
  ?SmallMoleculeRefXref rdf:type bp3:UnificationXref .
  ?SmallMoleculeRefXref bp3:db "ChEBI" .
  ?SmallMoleculeRefXref bp3:id ?SmallMoleculeID .

  FILTER (?SmallMoleculeID NOT IN ("CHEBI:15377", "CHEBI:24636", "CHEBI:15378", "CHEBI:15379",
  "CHEBI:57783","CHEBI:13392", "CHEBI:16474", "CHEBI:58349","CHEBI:18009", "CHEBI:13390","CHEBI:25523",
  "CHEBI:25524", "CHEBI:30616", "CHEBI:15422", "CHEBI:16761","CHEBI:456216", "CHEBI:57945",
  "CHEBI:16908","CHEBI:57540", "CHEBI:15846", "CHEBI:16526", "CHEBI:29888", "CHEBI:35782", "CHEBI:68836",
  "CHEBI:18361", "CHEBI:43474", "CHEBI:35780", "CHEBI:18367", "CHEBI:26078", "CHEBI:77740",
  "CHEBI:28931", "CHEBI:58307", "CHEBI:17877", "CHEBI:17552", "CHEBI:58189", "CHEBI:37565",
  "CHEBI:15996"))

  ?reaction bp3:right/(bp3:memberPhysicalEntity)* ?SmallMolecule2 .
  ?SmallMolecule2 rdf:type bp3:SmallMolecule .
  ?SmallMolecule2 bp3:cellularLocation ?cellularLocVocab2 .
  ?SmallMolecule2 bp3:entityReference ?SmallMoleculeRef .
  ?SmallMoleculeRef bp3:xref ?SmallMoleculeRefXref .
  ?SmallMoleculeRefXref rdf:type bp3:UnificationXref .
  ?SmallMoleculeRefXref bp3:db "ChEBI" .
  ?SmallMoleculeRefXref bp3:id ?SmallMoleculeID .

  ?catalysis bp3:controlled ?reaction .
  ?catalysis (rdf:type/rdfs:subClassOf*) bp3:Control .
  ?catalysis bp3:controller/((bp3:component|bp3:memberPhysicalEntity)*) ?enzyme .
  ?enzyme rdf:type bp3:Protein .
  ?enzyme bp3:entityReference ?enzymeRef .
  ?enzymeRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
  FILTER(?cellularLocVocab1 != ?cellularLocVocab2)
}
```

3.12 chemical-affects

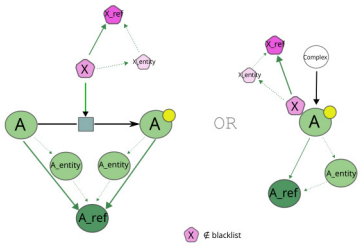
Paxtools pattern `chemicalAffectsProteinThroughBinding()` code

```
public static Pattern chemicalAffectsProteinThroughBinding(Blacklist blacklist)
{
    Pattern p = new Pattern(SmallMoleculeReference.class, "SMR");
    p.add(erToPE(), "SMR", "SPE1");
    p.add(notGeneric(), "SPE1");
    if (blacklist != null) p.add(new NonUbique(blacklist), "SPE1");
    p.add(linkToComplex(), "SPE1", "PE1");
    p.add(new PathConstraint("PhysicalEntity/componentOf"), "PE1", "Complex");
    p.add(new PathConstraint("Complex/component"), "Complex", "PE2");
    p.add(equal(false), "PE1", "PE2");
    p.add(linkToSpecific(), "PE2", "SPE2");
    p.add(new Type(SequenceEntity.class), "SPE2");
    p.add(peToER(), "SPE2", "generic-ER");
    p.add(linkedER(false), "generic-ER", "ER");
    return p;
}
```

Paxtools pattern `chemicalAffectsProteinThroughControl()` code

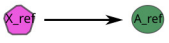
```
public static Pattern chemicalAffectsProteinThroughControl()
{
    Pattern p = new Pattern(SmallMoleculeReference.class, "controller-SMR");
    p.add(erToPE(), "controller-SMR", "controller-simple-PE");
    p.add(notGeneric(), "controller-simple-PE");
    p.add(linkToComplex(), "controller-simple-PE", "controller-PE");
    p.add(peToControl(), "controller-PE", "Control");
    p.add(controlToInter(), "Control", "Interaction");
    p.add(new NOT(participantER()), "Interaction", "controller-SMR");
    p.add(participant(), "Interaction", "affected-PE");
    p.add(linkToSpecific(), "affected-PE", "affected-simple-PE");
    p.add(new Type(SequenceEntity.class), "affected-simple-PE");
    p.add(peToER(), "affected-simple-PE", "affected-generic-ER");
    p.add(linkedER(false), "affected-generic-ER", "affected-ER");
    return p;
}
```

Sample BioPAX structure



OR

Inferred binary relation(s)



Meaning 3 SPARQL query

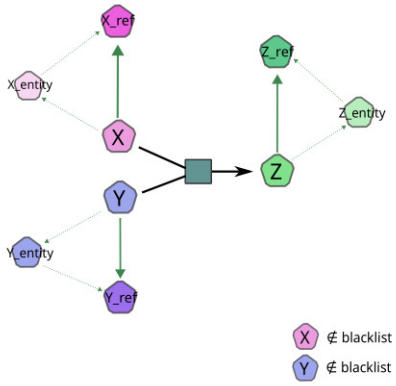
```
CONSTRUCT {
  ?chemicalCatalyzerRef abstraction:ChemicalAffects ?proteinRef .
}
WHERE {
  {
    ?complex rdf:type bp3:Complex .
    ?complex (bp3:component|bp3:memberPhysicalEntity)* ?proteinComponent .
    ?proteinComponent rdf:type bp3:Protein .
    # modification of the protein
    ?proteinComponent bp3:feature ?feature .
    ?feature rdf:type/(rdfs:subClassOf*) bp3:EntityFeature .
    ?complex (bp3:component|bp3:memberPhysicalEntity)* ?chemicalCatalyzer .
    ?chemicalCatalyzer rdf:type bp3:SmallMolecule .
    ?chemicalCatalyzer bp3:entityReference ?chemicalCatalyzerRef .
    ?chemicalCatalyzerRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
    ?chemicalCatalyzerRef bp3:xref ?chemicalCatalyzerRefXref .
    ?chemicalCatalyzerRefXref rdf:type bp3:UnificationXref .
    ?chemicalCatalyzerRefXref bp3:db "ChEBI" .
    ?chemicalCatalyzerRefXref bp3:id ?chemicalCatalyzerID .
    FILTER (?chemicalCatalyzerID NOT IN ("CHEBI:15377", "CHEBI:24636", "CHEBI:15378", "CHEBI:15379",
    "CHEBI:57783","CHEBI:13392", "CHEBI:16474", "CHEBI:58349", "CHEBI:18009","CHEBI:13390","CHEBI:25523",
    "CHEBI:25524", "CHEBI:30616", "CHEBI:15422", "CHEBI:16761","CHEBI:456216","CHEBI:57945", "CHEBI:16908",
    "CHEBI:57540", "CHEBI:15846","CHEBI:16526", "CHEBI:29888", "CHEBI:35782","CHEBI:68836", "CHEBI:18361",
    "CHEBI:43474","CHEBI:35780","CHEBI:18367","CHEBI:26078","CHEBI:77740","CHEBI:28931", "CHEBI:58307",
    "CHEBI:17877", "CHEBI:17552", "CHEBI:58189","CHEBI:37565", "CHEBI:15996"))
    BIND(?proteinComponent AS ?protein1)
  }
  UNION
  {
    ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
    ?reaction bp3:left/(bp3:memberPhysicalEntity)* ?protLeft .
    ?protLeft rdf:type bp3:Protein .
    ?reaction bp3:right/(bp3:memberPhysicalEntity)* ?protRight .
    ?protRight rdf:type bp3:Protein .
    ?catalysis bp3:controlled ?reaction .
    ?catalysis rdf:type/(rdfs:subClassOf*) bp3:Control .
    ?catalysis bp3:controller ?chemicalCatalyzer .
    ?chemicalCatalyzer rdf:type bp3:SmallMolecule .
    ?chemicalCatalyzer bp3:entityReference ?chemicalCatalyzerRef .
    ?chemicalCatalyzerRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
    ?chemicalCatalyzerRef bp3:xref ?chemicalCatalyzerRefXref .
    ?chemicalCatalyzerRefXref rdf:type bp3:UnificationXref .
    ?chemicalCatalyzerRefXref bp3:db "ChEBI" .
    ?chemicalCatalyzerRefXref bp3:id ?chemicalCatalyzerID .
    FILTER (?chemicalCatalyzerID NOT IN ("CHEBI:15377", "CHEBI:24636", "CHEBI:15378", "CHEBI:15379",
    "CHEBI:57783","CHEBI:13392", "CHEBI:16474", "CHEBI:58349", "CHEBI:18009","CHEBI:13390","CHEBI:25523",
    "CHEBI:25524", "CHEBI:30616", "CHEBI:15422", "CHEBI:16761","CHEBI:456216","CHEBI:57945", "CHEBI:16908",
    "CHEBI:57540", "CHEBI:15846","CHEBI:16526", "CHEBI:29888", "CHEBI:35782","CHEBI:68836", "CHEBI:18361",
    "CHEBI:43474","CHEBI:35780","CHEBI:18367","CHEBI:26078","CHEBI:77740","CHEBI:28931", "CHEBI:58307",
    "CHEBI:17877", "CHEBI:17552", "CHEBI:58189","CHEBI:37565", "CHEBI:15996"))
    # modification of the protein
    ?protRight bp3:feature ?feature .
    ?feature rdf:type/(rdfs:subClassOf*) bp3:EntityFeature .
    BIND(?protLeft AS ?protein2)
  }
  BIND(COALESCE(?protein1, ?protein2) AS ?protein)
  ?protein bp3:entityReference ?proteinRef .
  ?proteinRef rdf:type/(rdfs:subClassOf*) bp3:EntityReference .
}
```

3.13 reacts-with

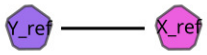
Paxtools pattern `reactsWith()` code

```
public static Pattern reactsWith(Blacklist blacklist)
{
    Pattern p = new Pattern(SmallMoleculeReference.class, "SMR1");
    p.add(erToPE(), "SMR1", "SPE1");
    p.add(notGeneric(), "SPE1");
    p.add(linkToComplex(blacklist), "SPE1", "PE1");
    p.add(new ParticipatesInConv(RelType.INPUT, blacklist), "PE1", "Conv");
    p.add(type(BiochemicalReaction.class), "Conv");
    p.add(new InterToPartER(InterToPartER.Direction.ONESIDERS), "Conv", "SMR1");
    p.add(new ConversionSide(ConversionSide.Type.SAME_SIDE, blacklist, RelType.INPUT),
    "PE1", "Conv", "PE2");
    p.add(type(SmallMolecule.class), "PE2");
    p.add(linkToSpecific(), "PE2", "SPE2");
    p.add(notGeneric(), "SPE2");
    p.add(new PEChainsIntersect(false), "SPE1", "PE1", "SPE2", "PE2");
    p.add(peToER(), "SPE2", "SMR2");
    p.add(equal(false), "SMR1", "SMR2");
    p.add(new InterToPartER(InterToPartER.Direction.ONESIDERS), "Conv", "SMR2");
    return p;
}
```

Sample BioPAX structure



Inferred binary relation(s)



Meaning 3 SPARQL query

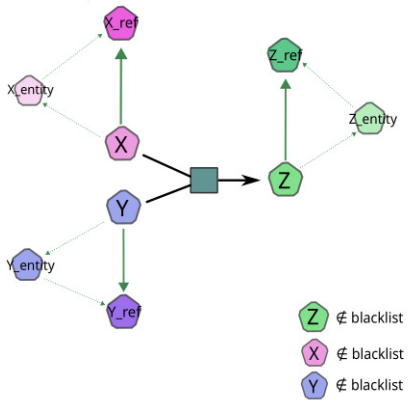
```
CONSTRUCT {
  ?smallMolecule1Ref abstraction:ReactsWith ?smallMolecule2Ref .
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:BiochemicalReaction .
  ?reaction bp3:left/(bp3:memberPhysicalEntity)* ?smallMolecule1 .
  ?reaction bp3:left/(bp3:memberPhysicalEntity)* ?smallMolecule2 .
  ?smallMolecule1 rdf:type bp3:SmallMolecule .
  ?smallMolecule1 bp3:entityReference ?smallMolecule1Ref .
  ?smallMolecule1Ref rdf:type bp3:SmallMoleculeReference .
  ?smallMolecule1Ref bp3:xref ?smallMolecule1RefXref .
  ?smallMolecule1RefXref rdf:type bp3:UnificationXref .
  ?smallMolecule1RefXref bp3:db "ChEBI" .
  ?smallMolecule1RefXref bp3:id ?smallMolecule1ID .
  FILTER (?smallMolecule1ID NOT IN ("CHEBI:15377", "CHEBI:24636", "CHEBI:15378", "CHEBI:15379",
    "CHEBI:57783","CHEBI:13392", "CHEBI:16474", "CHEBI:58349", "CHEBI:18009","CHEBI:13390","CHEBI:25523",
    "CHEBI:25524", "CHEBI:30616", "CHEBI:15422", "CHEBI:16761","CHEBI:456216","CHEBI:57945", "CHEBI:16908",
    "CHEBI:57540", "CHEBI:15846","CHEBI:16526", "CHEBI:29888", "CHEBI:35782","CHEBI:68836", "CHEBI:18361",
    "CHEBI:43474","CHEBI:35780","CHEBI:18367","CHEBI:26078","CHEBI:77740","CHEBI:28931", "CHEBI:58307",
    "CHEBI:17877", "CHEBI:17552", "CHEBI:58189","CHEBI:37565", "CHEBI:15996"))

  ?smallMolecule2 rdf:type bp3:SmallMolecule .
  ?smallMolecule2 bp3:entityReference ?smallMolecule2Ref .
  ?smallMolecule2Ref rdf:type bp3:SmallMoleculeReference .
  ?smallMolecule2Ref bp3:xref ?smallMolecule2RefXref .
  ?smallMolecule2RefXref rdf:type bp3:UnificationXref .
  ?smallMolecule2RefXref bp3:db "ChEBI" .
  ?smallMolecule2RefXref bp3:id ?smallMolecule2ID .
  FILTER (?smallMolecule2ID NOT IN ("CHEBI:15377", "CHEBI:24636", "CHEBI:15378", "CHEBI:15379",
    "CHEBI:57783","CHEBI:13392", "CHEBI:16474", "CHEBI:58349", "CHEBI:18009","CHEBI:13390","CHEBI:25523",
    "CHEBI:25524", "CHEBI:30616", "CHEBI:15422", "CHEBI:16761","CHEBI:456216","CHEBI:57945", "CHEBI:16908",
    "CHEBI:57540", "CHEBI:15846","CHEBI:16526", "CHEBI:29888", "CHEBI:35782","CHEBI:68836", "CHEBI:18361",
    "CHEBI:43474","CHEBI:35780","CHEBI:18367","CHEBI:26078","CHEBI:77740","CHEBI:28931", "CHEBI:58307",
    "CHEBI:17877", "CHEBI:17552", "CHEBI:58189","CHEBI:37565", "CHEBI:15996"))

  FILTER (?smallMolecule1Ref != ?smallMolecule2Ref)
}
```

3.14 used-to-produce

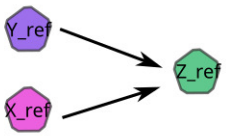
Paxtools pattern `usedToProduce()` code

```
public static Pattern usedToProduce(Blacklist blacklist)
{
    Pattern p = new Pattern(SmallMoleculeReference.class, "SMR1");
    p.add(erToPE(), "SMR1", "SPE1");
    p.add(notGeneric(), "SPE1");
    p.add(linkToComplex(blacklist), "SPE1", "PE1");
    p.add(new ParticipatesInConv(RelType.INPUT, blacklist), "PE1", "Conv");
    p.add(type(BiochemicalReaction.class), "Conv");
    p.add(new InterToPartER(InterToPartER.Direction.ONESIDERS), "Conv", "SMR1");
    p.add(new ConversionSide(ConversionSide.Type.OTHER_SIDE, blacklist, RelType.OUTPUT),
    "PE1", "Conv", "PE2");
    p.add(type(SmallMolecule.class), "PE2");
    p.add(linkToSimple(blacklist), "PE2", "SPE2");
    p.add(notGeneric(), "SPE2");
    p.add(equal(false), "SPE1", "SPE2");
    p.add(peToER(), "SPE2", "SMR2");
    p.add(equal(false), "SMR1", "SMR2");
    p.add(new InterToPartER(InterToPartER.Direction.ONESIDERS), "Conv", "SMR2");
    return p;
}
```

Sample BioPAX structure



Inferred binary relation(s)



Meaning 3 SPARQL query

```
CONSTRUCT {
  ?smallMolecule1Ref abstraction:UsedToProduce ?smallMolecule2Ref .
}
WHERE {
  ?reaction rdf:type/(rdfs:subClassOf*) bp3:Conversion .
  ?reaction bp3:left/(bp3:memberPhysicalEntity)* ?smallMolecule1 .
  ?reaction bp3:right/(bp3:memberPhysicalEntity)* ?smallMolecule2 .
  ?smallMolecule1 rdf:type bp3:SmallMolecule .
  ?smallMolecule1 bp3:entityReference ?smallMolecule1Ref .
  ?smallMolecule1Ref rdf:type bp3:SmallMoleculeReference .
  ?smallMolecule1Ref bp3:xref ?smallMolecule1RefXref .
  ?smallMolecule1RefXref rdf:type bp3:UnificationXref .
  ?smallMolecule1RefXref bp3:db "ChEBI" .
  ?smallMolecule1RefXref bp3:id ?smallMolecule1ID .
  FILTER (?smallMolecule1ID NOT IN ("CHEBI:15377", "CHEBI:24636", "CHEBI:15378", "CHEBI:15379",
    "CHEBI:57783","CHEBI:13392", "CHEBI:16474", "CHEBI:58349", "CHEBI:18009","CHEBI:13390","CHEBI:25523",
    "CHEBI:25524", "CHEBI:30616", "CHEBI:15422", "CHEBI:16761","CHEBI:456216","CHEBI:57945", "CHEBI:16908",
    "CHEBI:57540", "CHEBI:15846","CHEBI:16526", "CHEBI:29888", "CHEBI:35782","CHEBI:68836", "CHEBI:18361",
    "CHEBI:43474","CHEBI:35780","CHEBI:18367","CHEBI:26078","CHEBI:77740","CHEBI:28931", "CHEBI:58307",
    "CHEBI:17877", "CHEBI:17552", "CHEBI:58189","CHEBI:37565", "CHEBI:15996"))

  ?smallMolecule2 rdf:type bp3:SmallMolecule .
  ?smallMolecule2 bp3:entityReference ?smallMolecule2Ref .
  ?smallMolecule2Ref rdf:type bp3:SmallMoleculeReference .
  ?smallMolecule2Ref bp3:xref ?smallMolecule2RefXref .
  ?smallMolecule2RefXref rdf:type bp3:UnificationXref .
  ?smallMolecule2RefXref bp3:db "ChEBI" .
  ?smallMolecule2RefXref bp3:id ?smallMolecule2ID .
  FILTER (?smallMolecule2ID NOT IN ("CHEBI:15377", "CHEBI:24636", "CHEBI:15378", "CHEBI:15379",
    "CHEBI:57783","CHEBI:13392", "CHEBI:16474", "CHEBI:58349", "CHEBI:18009","CHEBI:13390","CHEBI:25523",
    "CHEBI:25524", "CHEBI:30616", "CHEBI:15422", "CHEBI:16761","CHEBI:456216","CHEBI:57945", "CHEBI:16908",
    "CHEBI:57540", "CHEBI:15846","CHEBI:16526", "CHEBI:29888", "CHEBI:35782","CHEBI:68836", "CHEBI:18361",
    "CHEBI:43474","CHEBI:35780","CHEBI:18367","CHEBI:26078","CHEBI:77740","CHEBI:28931", "CHEBI:58307",
    "CHEBI:17877", "CHEBI:17552", "CHEBI:58189","CHEBI:37565", "CHEBI:15996"))

  FILTER (?smallMolecule1Ref != ?smallMolecule2Ref)
}
```